

I.E.S. LA MARISMA — HUELVA

Departamento de Informática

C.F.G.S. Desarrollo de Aplicaciones Multiplataforma

SLIOR

Un Sistema Logístico Inteligente de Optimización de Rutas

Autor: José Ramos Contioso

Tutor/a: Fiallega Escauriaza, María Arantzazu

Fecha: Mayo 2026

HOJA RESUMEN DEL PROYECTO

Título del proyecto:	SLIOR — Un Sistema Logístico Inteligente de Optimización de Rutas
Autor:	José Ramos Contioso
Fecha:	Mayo 2026
Tutor/a:	[Nombre de la tutora]
Titulación:	C.F.G.S. Desarrollo de Aplicaciones Multiplataforma
Palabras clave (ES):	logística, optimización de rutas, Android, Spring Boot, offline-first, RGPD, OpenStreetMap
Keywords (EN):	logistics, route optimisation, Android, Spring Boot, offline-first, GDPR, OpenStreetMap

Resumen (Español)

SLIOR es un sistema integral de gestión y optimización de rutas de reparto compuesto por una API REST desarrollada con Spring Boot y una aplicación móvil nativa para Android desarrollada con Kotlin y Jetpack Compose. El sistema permite a los repartidores gestionar sus rutas de entrega, optimizar el orden de las paradas mediante el algoritmo del vecino más cercano con cálculo real de distancias a través de OSRM, confirmar entregas mediante escaneo de códigos de barras y QR, y seguir trabajando sin conexión gracias a una arquitectura offline-first basada en Room y WorkManager con reintentos con backoff exponencial. El backend implementa autenticación JWT stateless, rate limiting por IP mediante Bucket4j, auditoría de acciones sensibles con hash SHA-256 y cumplimiento íntegro del Reglamento General de Protección de Datos (RGPD). La infraestructura completa se orquesta mediante Docker Compose y se apoya exclusivamente en software de código abierto, eliminando dependencias de servicios externos de pago.

Abstract (English)

SLIOR is a comprehensive delivery route management and optimisation system consisting of a REST API built with Spring Boot and a native Android mobile application built with Kotlin and Jetpack Compose. The system allows couriers to manage their delivery routes, optimise stop order using the nearest-neighbour algorithm with real road distance calculation via OSRM, confirm deliveries by scanning barcodes and QR codes, and continue working offline thanks to an offline-first architecture based on Room and WorkManager with exponential backoff retries. The backend implements stateless JWT authentication, per-IP rate limiting via Bucket4j,

sensitive action auditing with SHA-256 hashing, and full compliance with the General Data Protection Regulation (GDPR). The entire infrastructure is orchestrated with Docker Compose and relies exclusively on open-source software, removing dependencies on paid external services.

ÍNDICE

HOJA RESUMEN DEL PROYECTO	2
ÍNDICE	4
1. INTRODUCCIÓN	8
1.1. INTRODUCCIÓN A LA MEMORIA	8
1.2. DESCRIPCIÓN DEL PROYECTO	8
1.3. OBJETIVOS GENERALES	9
1.4. BENEFICIOS DEL SISTEMA	9
1.5. MOTIVACIONES PERSONALES	10
1.6. ESTRUCTURA DE LA MEMORIA	10
2. ESTUDIO DE VIABILIDAD	12
2.1. INTRODUCCIÓN	12
2.1.1. Tipología y palabras clave	12
2.1.2. Descripción del proyecto	12
2.1.3. Objetivos del proyecto	12
2.1.4. Clasificación de los objetivos	13
2.1.5. Definiciones, acrónimos y abreviaciones	13
2.1.6. Partes interesadas	15
2.1.7. Referencias bibliográficas utilizadas en la viabilidad	15
2.2. ESTUDIO DE LA SITUACIÓN ACTUAL	16
2.2.1. Contexto del sector logístico de última milla	16
2.2.2. Descripción del sistema actual	17
2.2.3. Diagnóstico del sistema actual	17
2.2.4. Normativa y legislación aplicable	18
2.3. REQUISITOS DEL SISTEMA	19
2.3.1. Requisitos funcionales	19
2.3.2. Requisitos no funcionales	21
2.3.3. Restricciones del sistema	22
2.3.4. Catalogación y priorización de los requisitos	23
2.4. ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN	24
2.4.1. Alternativa 1: Aplicación web progresiva (PWA)	24
2.4.2. Alternativa 2: Aplicación multiplataforma con Flutter	25
2.4.3. Alternativa 3: Aplicación nativa Android con Kotlin — SELECCIONADA	25
2.4.4. Tabla comparativa de alternativas	26
2.5. PLANIFICACIÓN DEL PROYECTO	26
2.5.1. Metodología de desarrollo	26
2.5.2. Recursos del proyecto	27
2.5.3. División del trabajo en fases	27
2.5.4. Estimación de tiempos	30
2.5.5. Planificación temporal	30
2.6. EVALUACIÓN DE RIESGOS	31

2.6.1. Identificación de riesgos	31
2.6.2. Plan de contingencia	32
2.7. PRESUPUESTO	32
2.7.1. Estimación de coste material	32
2.7.2. Estimación de coste personal	33
2.7.3. Resumen y análisis coste-beneficio	34
2.8. CONCLUSIONES DEL ESTUDIO DE VIABILIDAD	34
3. ANÁLISIS	35
3.1. INTRODUCCIÓN	35
3.2. CATÁLOGO DE ACTORES	35
3.3. HISTORIAS DE USUARIO	35
HU-01 — Registro de usuario	35
HU-02 — Inicio de sesión	36
HU-03 — Consultar mis rutas del día	36
HU-04 — Optimizar el orden de entregas	37
HU-05 — Confirmar entrega offline	37
HU-06 — Navegar a la siguiente parada	37
HU-07 — Exportar mis datos personales (RGPD)	38
HU-08 — Cambiar el idioma de la aplicación	38
3.4. REQUISITOS FUNCIONALES DE USUARIOS	38
3.5. REQUISITOS NO FUNCIONALES — RESTRICCIONES DE DISEÑO	39
3.6. DIAGRAMAS DE CASOS DE USO	40
3.6.1. Diagrama de casos de uso — Autenticación	40
3.6.2. Diagrama de casos de uso — Gestión de rutas	40
3.6.3. Diagrama de casos de uso — Gestión de entregas	41
3.6.4. Diagrama de casos de uso — RGPD y privacidad	41
3.6.5. Especificación de casos de uso principales	41
3.7. MODELO LÓGICO DE DATOS	44
3.8. DIAGRAMA DE CLASES DEL BACKEND	45
3.9. DIAGRAMA DE CLASES DE LA APLICACIÓN ANDROID	45
3.10. DIAGRAMAS DE INTERACCIÓN	46
3.10.1. Diagrama de secuencia — Flujo de inicio de sesión	46
3.10.2. Diagrama de secuencia — Flujo de confirmación offline y sincronización	46
3.10.3. Diagrama de secuencia — Flujo de optimización de ruta	47
3.11. MAPA DE NAVEGACIÓN DE LA APLICACIÓN ANDROID	47
3.12. CONCLUSIÓN DEL ANÁLISIS	48
4. DISEÑO	49
4.1. INTRODUCCIÓN	49
4.2. CONFIGURACIÓN DE LA PLATAFORMA TECNOLÓGICA	49
4.2.1. Entorno de desarrollo backend	49
4.2.2. Entorno de desarrollo Android	50
4.3. ARQUITECTURA DEL SISTEMA	51
4.4. CAPAS DE LA APLICACIÓN	52

4.4.1. Capas del backend (Spring Boot)	52
4.4.2. Capas de la aplicación Android (MVVM + Repository)	53
4.5. DISEÑO DE LA BASE DE DATOS	54
4.5.1. Esquema de la base de datos PostgreSQL	54
4.6. DISEÑO DE LA API REST	55
4.7. DISEÑO DE LA INTERFAZ DE USUARIO ANDROID	57
4.7.1. Sistema de diseño — Estilo Neo-Brutalista	57
4.7.2. Pantallas principales	59
5. IMPLEMENTACIÓN	66
5.1. INTRODUCCIÓN	66
5.2. BACKEND — CODIFICACIÓN	66
5.2.1. Estructura del proyecto Spring Boot	66
5.2.2. Autenticación JWT con Spring Security 6	67
5.2.3. Algoritmo del vecino más cercano (Nearest Neighbor)	67
5.2.4. Geocodificación híbrida de tres niveles	68
5.2.5. Generación de etiquetas PDF con OpenPDF y ZXing	68
5.2.7. Rate limiting con Bucket4j	69
5.3. APLICACIÓN ANDROID — CODIFICACIÓN	69
5.3.1. Estructura del proyecto Android	69
Esta estructura refleja una arquitectura MVVM + Repository con Hilt, Room, Retrofit y WorkManager.	70
5.3.2. Arquitectura offline-first con Room y StateFlow	70
5.3.3. SyncDeliveryWorker con reintentos	70
5.3.4. Integración de OSMDroid con Jetpack Compose	71
5.3.5. Internacionalización con AppCompatDelegate	71
5.4. INFRAESTRUCTURA — DOCKER COMPOSE	71
6. PRUEBAS	74
6.1. INTRODUCCIÓN	74
6.2. PLAN DE PRUEBAS	74
6.2.1. Tipos de prueba y herramientas	74
6.3. PRUEBAS UNITARIAS DEL BACKEND	74
6.4. PRUEBAS DE INTEGRACIÓN DE LA API	75
6.4.1. Pruebas del subsistema de autenticación	75
6.4.2. Pruebas del subsistema de rutas y optimización	75
6.5. PRUEBAS FUNCIONALES DE LA APLICACIÓN ANDROID	76
6.5.1. Pruebas de visualización	76
6.5.2. Prueba PF-01 — Flujo completo de registro y login	77
6.5.3. Prueba PF-02 — Creación y optimización de ruta	78
6.5.4. Prueba PF-03 — Exportación de datos RGPD	78
6.6. RESULTADOS GLOBALES	79
6.7. CONCLUSIONES DE LAS PRUEBAS	79
7. CONCLUSIONES	81
7.1. CONCLUSIONES FINALES	81
7.2. DESVIACIONES TEMPORALES	81

7.3. POSIBLES AMPLIACIONES Y MODIFICACIONES	82
7.4. VALORACIÓN PERSONAL	82
8. BIBLIOGRAFÍA	84
9. GLOSARIO	86
10. ANEXOS	89
ANEXO A — REGISTRO DE ERRORES Y SOLUCIONES	89
Error A-01: La app Android no conecta con la API en el emulador	89
Error A-02: Incompatibilidad de OpenPDF con Java 17	89
Error A-03: Desbordamiento de la UI en modo horizontal	89
Error A-04: Conflictos de tipos en Room TypeConverters	90
Error A-05: Bloqueo del hilo principal en la generación de PDF	90
Error A-06: Código de barras Code 128 no detectado por el escáner del smartphone	90
Error A-07: Ciclo de vida de OSMDroid en Compose	90
ANEXO B — LICENCIAS DEL SOFTWARE UTILIZADO	91
ANEXO C — MANUAL DE INSTALACIÓN Y DESPLIEGUE	92
C.1. Requisitos del sistema	92
C.2. Pasos de instalación del backend	93
C.3. Instalación de la app Android	93

1. INTRODUCCIÓN

1.1. INTRODUCCIÓN A LA MEMORIA

El presente documento recoge la memoria técnica del proyecto **SLIOR**, desarrollado como trabajo final del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma del IES La Marisma (Huelva). El proyecto consiste en el diseño, desarrollo y despliegue de un sistema completo de gestión y optimización de rutas logísticas, orientado a facilitar el trabajo diario de los repartidores de pequeñas empresas y autónomos del sector logístico de última milla.

La memoria sigue la estructura establecida en las normas del módulo Proyecto Intermodular del Departamento de Informática del IES La Marisma e incluye, en este orden, el estudio de viabilidad, el análisis, el diseño, la implementación y las pruebas del sistema. Se han utilizado diagramas UML siempre que ha resultado pertinente para la comprensión del sistema, así como tablas comparativas para justificar las decisiones tecnológicas adoptadas.

Este proyecto es de elaboración individual. El alumno ha asumido los roles de analista, diseñador, programador y tester a lo largo de todo el ciclo de vida del desarrollo.

1.2. DESCRIPCIÓN DEL PROYECTO

SLIOR (Sistema Logístico Inteligente de Optimización de Rutas) es una solución tecnológica compuesta por dos componentes principales que trabajan de forma coordinada:

- API REST backend: desarrollada en Java 17 con el framework Spring Boot 3.2.4, gestiona toda la lógica de negocio del sistema: usuarios, autenticación, rutas, paradas, geocodificación, generación de etiquetas logísticas en formato PDF y cumplimiento del RGPD. Se persiste en PostgreSQL 14 y el esquema de base de datos se gestiona con Flyway. El acceso está protegido por Spring Security con autenticación JWT stateless y rate limiting por IP.
- Aplicación móvil Android: desarrollada en Kotlin 2.0.21 con Jetpack Compose (Material 3) siguiendo la arquitectura MVVM con repositorios. Permite al repartidor consultar y gestionar sus rutas, visualizarlas sobre un mapa OpenStreetMap renderizado con OSMDroid, confirmar entregas mediante escaneo de códigos de barras y QR con ZXing, y trabajar sin conexión gracias a una capa de persistencia local basada en Room con sincronización automática a través de WorkManager.

Ambos componentes se comunican exclusivamente a través de la API REST. El despliegue se realiza mediante Docker Compose sobre infraestructura local o de red privada, sin dependencias de servicios cloud

externos de pago. Todo el stack tecnológico empleado es software de código abierto.

El proyecto nace de la observación de que los sistemas logísticos accesibles para pequeñas empresas o autónomos suelen ser costosos, propietarios o demasiado complejos para su adopción. La elección de software libre y la integración con herramientas ya presentes en el flujo de trabajo habitual del sector, como Google Maps para la navegación paso a paso, buscan reducir la barrera de adopción y minimizar los costes operativos para el usuario final.

1.3. OBJETIVOS GENERALES

Los objetivos principales del proyecto son los siguientes:

- Proporcionar a los repartidores una herramienta móvil intuitiva que centralice la gestión de sus rutas de entrega, reduciendo el tiempo dedicado a la planificación manual.
- Implementar un algoritmo de optimización del orden de paradas que minimice la distancia total recorrida, con cálculo real de tiempos mediante OSRM.
- Garantizar la continuidad del trabajo en entornos sin cobertura de red mediante una arquitectura offline-first con sincronización automática diferida.
- Ofrecer compatibilidad con las herramientas de navegación ya utilizadas por los repartidores (Google Maps, Waze), facilitando la transición al nuevo sistema sin romper sus flujos de trabajo habituales.
- Cumplir la normativa vigente en materia de protección de datos (RGPD / LOPDGDD), implementando los derechos ARSULIPO y un sistema de auditoría de acciones sensibles.
- Construir el sistema íntegramente sobre software de código abierto, eliminando dependencias de servicios externos de pago y reduciendo el coste total de operación.
- Ofrecer una interfaz multilingüe (cinco idiomas) para facilitar la adopción en entornos multiculturales.

1.4. BENEFICIOS DEL SISTEMA

La implantación de **SLIOR** reporta los siguientes beneficios directos e indirectos:

- Reducción del tiempo de planificación de rutas al automatizar el cálculo del orden mejorado de paradas mediante el algoritmo del vecino más cercano (Nearest Neighbor), reduciendo la distancia total sin incurrir en el coste computacional de resolver el Problema del Viajante.
- Ahorro de combustible y desgaste del vehículo gracias a rutas de menor distancia total.

- Eliminación del papel en el proceso de entrega: las etiquetas se generan digitalmente y la confirmación se realiza mediante escaneo de código, sin necesidad de albaranes físicos.
- Mayor resiliencia operativa al permitir el registro de entregas sin conexión, con sincronización diferida transparente para el usuario.
- Trazabilidad completa de las entregas gracias al registro de auditoría, las marcas temporales de entrega y los logs del sistema.
- Coste de operación reducido al no depender de APIs de cartografía o geocodificación de pago (Google Maps Platform, HERE, etc.). Para un entorno de producción real se recomienda la auto-hospedaje de una instancia OSRM, ya que en este momento se usa la API demo, que tiene grandes limitaciones.
- Facilidad de adopción al integrarse con Google Maps para la navegación, herramienta ya familiar para la mayoría de repartidores.
- Cumplimiento normativo que evita sanciones derivadas del incumplimiento del RGPD.

1.5. MOTIVACIONES PERSONALES

La motivación principal para desarrollar **SLIOR** fue la observación de primera mano de las dificultades que enfrentan los repartidores de pequeñas empresas locales a la hora de planificar sus jornadas. El proceso habitual —lista en papel, Google Maps abierto para cada dirección por separado, llamadas de teléfono para confirmar entregas— es ineficiente, propenso a errores y difícil de supervisar para el gestor.

Se identificó una brecha de mercado clara: las soluciones profesionales de optimización de rutas (Route4Me, OptimoRoute, Circuit) son excelentes técnicamente pero tienen un coste mensual inasumible para autónomos o microempresas, con planes de suscripción que superan habitualmente los 30–100 €/mes por usuario. Las alternativas gratuitas o de bajo coste son demasiado genéricas y no cubren el flujo completo de trabajo de un repartidor (creación de ruta, navegación, escaneo, confirmación offline).

Desde el punto de vista formativo, el proyecto permitió aplicar de forma integrada las principales áreas del ciclo: programación backend con Java y Spring Boot, desarrollo Android nativo con Kotlin y Jetpack Compose, bases de datos relacionales con PostgreSQL, seguridad web con JWT y Spring Security, contenedores con Docker y cumplimiento normativo en protección de datos personales (RGPD/LOPDGDD).

1.6. ESTRUCTURA DE LA MEMORIA

La presente memoria se organiza en los capítulos siguientes:

- Capítulo 1 — Introducción: descripción general del proyecto, objetivos, beneficios y motivaciones.
- Capítulo 2 — Estudio de viabilidad: análisis del contexto, requisitos, alternativas tecnológicas con análisis comparativo, alternativas de

despliegue e infraestructura, planificación temporal, evaluación de riesgos y presupuesto.

- Capítulo 3 — Análisis: catálogo de actores, historias de usuario, requisitos funcionales y no funcionales, casos de uso, modelo de datos con diagrama Entidad-Relación, diagramas de clases del backend y la app Android, y diagramas de secuencia para los flujos principales.
- Capítulo 4 — Diseño: arquitectura del sistema, diseño de la base de datos, diseño de la API REST, diseño de la interfaz móvil y justificación de decisiones de diseño.
- Capítulo 5 — Implementación: detalles técnicos de la codificación del backend y la app Android, fragmentos de código explicados y decisiones de implementación.
- Capítulo 6 — Pruebas: plan de pruebas, pruebas unitarias, de integración, funcionales y no funcionales, y resultados.
- Capítulo 7 — Conclusiones: valoración del resultado, desviaciones temporales y propuestas de ampliación.
- Bibliografía, Glosario y Anexos (incluyendo registro de actividades de tratamiento y política de privacidad de la aplicación).

2. ESTUDIO DE VIABILIDAD

2.1. INTRODUCCIÓN

2.1.1. Tipología y palabras clave

Tipo de proyecto: Creación de aplicación — sistema cliente-servidor con arquitectura MVVM en el componente móvil Android y API REST stateless en el backend, íntegramente construido sobre software de código abierto.

Palabras clave: logística, optimización de rutas, vecino más cercano (Nearest Neighbor), Android nativo, Kotlin, Jetpack Compose, Spring Boot, JWT, arquitectura offline-first, escaneo de códigos de barras, Docker, OpenStreetMap, OSRM, PostgreSQL, RGPD.

2.1.2. Descripción del proyecto

SLIOR es un sistema de información logístico pensado para empresas de reparto de pequeño y mediano tamaño, así como para repartidores autónomos. El sistema cubre el ciclo completo de una jornada de reparto: creación de la ruta por el propio repartidor desde la app móvil (el rol administrador gestiona rutas a través de la API REST), optimización automática del orden de paradas, navegación hacia cada destino (con integración opcional de Google Maps y Waze), confirmación de entrega mediante escaneo de código QR o de barras del paquete, y sincronización de todos los eventos con el servidor central, incluso cuando el dispositivo ha estado sin cobertura.

El sistema se diferencia de las soluciones comerciales existentes en tres aspectos clave: uso exclusivo de software de código abierto (sin costes de licencia), arquitectura offline-first que garantiza la operatividad en zonas sin cobertura, y compatibilidad deliberada con Google Maps para reducir la resistencia al cambio por parte de los usuarios finales.

2.1.3. Objetivos del proyecto

Se definen los siguientes objetivos específicos y medibles, numerados para su trazabilidad a lo largo de la memoria:

ID	Objetivo	Prioridad
OBJ-01	Implementar una API REST que gestione usuarios, rutas y paradas con autenticación JWT stateless.	Alta
OBJ-02	Desarrollar un algoritmo de optimización de rutas (vecino más cercano) con cálculo real de distancias mediante OSRM.	Alta
OBJ-03	Crear una aplicación Android que funcione sin conexión y sincronice automáticamente al recuperarla, con reintentos con backoff exponencial.	Alta

OBJ-04	Integrar un sistema de escaneo de códigos de barras (Code 128) y QR para la confirmación de entregas.	Alta
OBJ-05	Implementar la generación automática de etiquetas logísticas en PDF con código de barras y código QR.	Media
OBJ-06	Cumplir el RGPD implementando exportación JSON, borrado lógico, anonimización automática y registro de auditoría.	Alta
OBJ-07	Desplegar el sistema mediante Docker Compose sin dependencias de servicios de pago.	Alta
OBJ-08	Ofrecer la interfaz Android en cinco idiomas (ES, EN, FR, PT, DE).	Media
OBJ-09	Implementar rate limiting por IP en los endpoints públicos para mitigar ataques de fuerza bruta y abuso de la API	Media

2.1.4. Clasificación de los objetivos

Los objetivos OBJ-01 a OBJ-04, OBJ-06 y OBJ-07 conforman el núcleo mínimo viable del sistema. Los objetivos OBJ-05, OBJ-08 y OBJ-09 son de valor añadido. En caso de desviación temporal crítica, serían los primeros en reducirse en alcance.

2.1.5. Definiciones, acrónimos y abreviaciones

Término	Definición
API REST	Application Programming Interface basada en el estilo arquitectónico REST (Representational State Transfer), que usa HTTP y sus verbos (GET, POST, PUT, PATCH, DELETE) para la comunicación sin estado.
Backoff exponencial	Estrategia de reintento en la que el tiempo de espera entre intentos sucesivos crece de forma exponencial, reduciendo la presión sobre el servidor en situaciones de error transitorio.
BCrypt	Algoritmo de hashing de contraseñas basado en el cifrado Blowfish, diseñado para ser computacionalmente costoso y resistente a ataques de fuerza bruta y diccionario.
Borrado lógico	Técnica que marca un registro como eliminado (campo is_deleted) en lugar de borrarlo físicamente de la base de datos, preservando la integridad histórica y el registro de auditoría.
DAO / Repository	Data Access Object / Repository. Patrón de diseño que abstrae y encapsula el acceso a la fuente de datos.

DTO	Data Transfer Object. Objeto usado para transferir datos entre capas de la aplicación sin exponer las entidades internas.
Geocodificación	Proceso de convertir una dirección postal textual en coordenadas geográficas (latitud/longitud) para su uso en sistemas de mapas.
Haversine	Fórmula trigonométrica que calcula la distancia entre dos puntos sobre la superficie de una esfera (la Tierra) a partir de sus coordenadas geográficas.
JWT	JSON Web Token. Estándar abierto (RFC 7519) para transmitir información de autenticación de forma segura entre partes como un objeto JSON firmado digitalmente con HMAC o RSA.
MVVM	Model-View-ViewModel. Patrón de arquitectura de software que separa la lógica de presentación (ViewModel) de la interfaz gráfica (View) y de los datos (Model).
Offline-first	Filosofía de diseño de aplicaciones en la que el dispositivo funciona correctamente sin conexión a internet, y sincroniza los cambios de forma diferida cuando recupera la conectividad.
ORM	Object-Relational Mapping. Técnica que mapea automáticamente objetos de una aplicación orientada a objetos a filas de tablas de una base de datos relacional.
OSRM	Open Source Routing Machine. Motor de cálculo de rutas por carretera de código abierto, basado en datos de OpenStreetMap. Por defecto calcula la ruta más rápida entre dos puntos considerando los límites de velocidad de cada vía.
Rate limiting	Técnica para controlar y limitar la tasa de peticiones que un cliente puede realizar a un servicio en un intervalo de tiempo determinado, para prevenir abusos y ataques.
RGPD / GDPR	Reglamento General de Protección de Datos de la Unión Europea (Reglamento 2016/679). Norma que regula el tratamiento de datos personales de personas físicas en la UE.
Room	Librería de persistencia de Android que proporciona una capa de abstracción sobre SQLite, facilitando el acceso a bases de datos locales con un API limpia.
SHA-256	Función hash criptográfica de la familia SHA-2 que produce un resumen de 256 bits. En SLIOR se usa para anonimizar las direcciones IP antes de registrarlas en el log de auditoría.
UUID	Universally Unique Identifier. Identificador de 128 bits que garantiza unicidad global sin necesidad de un servidor de coordinación.
Vecino más cercano (Nearest Neighbor)	Heurística greedy de optimización de rutas que, partiendo de un punto inicial, selecciona iterativamente la parada no visitada más próxima. No garantiza la solución óptima global pero ofrece un resultado razonablemente bueno con coste computacional lineal.

WorkManager	API de Android Jetpack que garantiza la ejecución de tareas en segundo plano, incluso si la aplicación es cerrada o el dispositivo reiniciado.
-------------	--

2.1.6. Partes interesadas

Rol	Perfil	Interés en el sistema
Repartidor	Usuario operativo, baja competencia técnica, usa el dispositivo con las manos ocupadas.	Usar la app de forma simple, sin papeleo y con soporte offline.
Administrador de flota	Gestor con acceso de escritura. En esta versión accede a través de la misma app móvil; en una versión futura dispondría de un panel de gestión dedicado.	Reducir tiempo de planificación y tener trazabilidad de las entregas.
Empresa logística	Cliente del sistema. Dueño del negocio.	Reducir costes operativos, eliminar papel y cumplir con la normativa vigente (RGPD).
Destinatarios	Personas que reciben los paquetes.	Recibir la entrega en el horario previsto; el sistema les afecta indirectamente al mejorar la puntualidad del repartidor.
Desarrollador	José Ramos Contoso, autor del proyecto.	Completar el TFG con un sistema funcional y bien documentado.

2.1.7. Referencias bibliográficas utilizadas en la viabilidad

- VMware (Broadcom). (2024). Spring Boot Reference Documentation 3.2. Recuperado de <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- Google. (2024). Guide to app architecture. Android Developers. Recuperado de <https://developer.android.com/topic/architecture>
- Google. (2024). Jetpack Compose. Android Developers. Recuperado de <https://developer.android.com/jetpack/compose>
- Google. (2024). Room Persistence Library. Android Developers. Recuperado de <https://developer.android.com/training/data-storage/room>
- Google. (2024). WorkManager. Android Developers. Recuperado de <https://developer.android.com/topic/libraries/architecture/workmanager>

- OSMDroid Contributors. (2024). osmdroid — OpenStreetMap tools for Android [Repositorio GitHub]. Recuperado de <https://github.com/osmdroid/osmdroid>
- Project OSRM. (2024). OSRM API Documentation v5.24. Recuperado de <http://project-osrm.org/docs/v5.24.0/api/>
- OpenStreetMap Foundation. (2024). Nominatim documentation. Recuperado de <https://nominatim.org/release-docs/latest/>
- Parlamento Europeo y Consejo de la UE. (2016). Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD). Diario Oficial de la Unión Europea. Recuperado de <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>
- Jefatura del Estado. (2018). Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD). BOE núm. 294. Recuperado de <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- Docker Inc. (2024). Docker Documentation. Recuperado de <https://docs.docker.com/>
- Kovalev, V. (2024). Bucket4j — Java rate-limiting library [Repositorio GitHub]. Recuperado de <https://github.com/bucket4j/bucket4j>
- Andres, B., & Micah, S. (2024). jwt — Java JWT library [Repositorio GitHub]. Recuperado de <https://github.com/jwt/jwt>
- CNMC. (2024). Informe Anual del Sector Postal 2023 (INF/DTSP/039/24). Comisión Nacional de los Mercados y la Competencia. Recuperado de <https://www.cnmc.es/prensa/informe-anual-postal-2023-20240724>

2.2. ESTUDIO DE LA SITUACIÓN ACTUAL

2.2.1. Contexto del sector logístico de última milla

El sector logístico de última milla (last-mile delivery) es el segmento de la cadena de suministro que cubre el trayecto final desde el punto de distribución hasta el destinatario. En los últimos años ha experimentado un crecimiento sostenido impulsado por el auge del comercio electrónico: el volumen de envíos de paquetería en España en 2022 fue un 93% superior al registrado en 2019, según el Informe Anual del Sector Postal de la CNMC.

Este crecimiento ha generado una demanda de herramientas de gestión de rutas accesibles para operadores de pequeño tamaño (empresas de 1 a 10 vehículos y autónomos), que representan la mayoría de los actores en el segmento de entrega local. Sin embargo, las soluciones tecnológicas disponibles presentan importantes barreras de acceso para este segmento:

- Coste elevado: las plataformas SaaS de referencia (Route4Me, OptimoRoute, Circuit) tienen planes de pago que oscilan entre 20€ y

100€ mensuales por conductor, lo que supone un gasto significativo para una microempresa.

- Curva de aprendizaje: muchas plataformas requieren un proceso de onboarding complejo y asistencia técnica para su implantación.
- Dependencia de servicios cloud: requieren conexión permanente a internet para funcionar, lo que es un problema en zonas industriales o rurales con cobertura limitada.
- Ausencia de integración con herramientas ya usadas: los repartidores tienen que abandonar completamente sus herramientas habituales (Google Maps, WhatsApp), generando resistencia al cambio.

2.2.2. Descripción del sistema actual

El flujo de trabajo típico de una microempresa de reparto local o autónomo de última milla, antes de la implantación de un sistema de gestión como **SLIOR**, implica las siguientes etapas:

1. Un gestor elabora manualmente la lista de entregas del día en papel o en una hoja de cálculo (normalmente Microsoft Excel u OpenOffice Calc).
2. El repartidor recibe la lista impresa o por WhatsApp y planifica mentalmente el orden de visitas basándose en su conocimiento de la zona.
3. La navegación se realiza con Google Maps, introduciendo manualmente cada dirección de destino de forma individual antes de cada entrega.
4. La confirmación de entrega se comunica al gestor mediante una llamada telefónica o un mensaje de WhatsApp al finalizar cada parada.
5. Al final del día, el gestor actualiza manualmente el estado de cada paquete en su registro (papel o Excel).
6. Los paquetes no entregados se gestionan de forma reactiva: el repartidor notifica la incidencia al gestor por teléfono o WhatsApp, sin registro formal del motivo ni del intento de entrega.

Este proceso carece por completo de: optimización automática de rutas, trazabilidad en tiempo real, registro formal de entregas con evidencia, soporte offline estructurado y cumplimiento normativo en el tratamiento de datos de destinatarios.

2.2.3. Diagnóstico del sistema actual

Problema identificado	Impacto
Orden de entregas no optimizado	El repartidor recorre más kilómetros de los necesarios, aumentando el gasto en combustible y el tiempo de jornada.

Falta de trazabilidad y registro formal	No existe evidencia digital de las entregas realizadas ni de los intentos fallidos. En caso de disputa con un cliente, no hay registro del intento de entrega ni del motivo de la incidencia.
Dependencia del soporte físico	Los albaranes en papel se pierden o deterioran. La información de entregas no es recuperable ni consultable a posteriori.
Dependencia de conexión / Ausencia de soporte offline	En zonas sin cobertura (polígonos industriales, áreas rurales), el repartidor no puede acceder a la información de la ruta ni registrar entregas, paralizando el trabajo.
Comunicación por canales no estructurados	El uso de WhatsApp mezcla información personal y profesional, no deja registro estructurado y hace imposible el control o la auditoría del proceso.
Incumplimiento potencial RGPD	Los datos personales de los destinatarios (nombre, dirección, teléfono) se transmiten por canales no cifrados y se almacenan sin medidas de protección adecuadas, incurriendo en riesgo normativo.
Tiempo de gestión no optimizado	La preparación manual de rutas y el registro posterior de resultados consumen tiempo diario que podría automatizarse, limitando la capacidad operativa del negocio.
Ausencia de herramienta integradora	Cada parte del proceso se gestiona con una herramienta distinta (Excel, WhatsApp, Google Maps) sin visión unificada, generando fricciones, errores y pérdida de información entre herramientas

2.2.4. Normativa y legislación aplicable

El sistema gestiona datos personales de dos categorías de interesados: por un lado, los destinatarios de los paquetes (nombre completo, dirección postal y teléfono de contacto); por otro, los propios repartidores (nombre, email y datos de actividad registrados durante el uso de la aplicación). Por este motivo, el sistema está sujeto a la normativa siguiente:

- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016 (RGPD). Norma de referencia en materia de protección de datos personales en la Unión Europea. Aplicable por tratarse de un sistema que recoge, almacena y trata datos de personas físicas identificables.
- Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD). Transposición española del RGPD que complementa y concreta su aplicación en el ordenamiento jurídico nacional.
- Ley 34/2002, de 11 de julio, de Servicios de la Sociedad de la Información y de Comercio Electrónico (LSSI-CE). Aplicable por tratarse de un servicio prestado por vía electrónica a través de una API REST con usuarios registrados.

SLIOR implementa los mecanismos técnicos necesarios para dar cumplimiento a los derechos reconocidos por el RGPD y la LOPDGDD:

Derecho		Implementación en SLIOR
Derecho de acceso		Endpoint GET /api/v1/users/me/data-export que devuelve todos los datos del usuario en formato JSON.
Derecho de rectificación	de	El usuario puede actualizar su perfil y contraseña desde la aplicación móvil.
Derecho de supresión	de	Borrado lógico inmediato (isDeleted = true) con anonimización automática de datos personales transcurridos 7 días, ejecutada mediante tarea programada (@Scheduled).
Derecho a la portabilidad	la	Los datos se exportan en formato JSON estándar, legible por máquina y transferible a otro sistema.
Limitación del tratamiento	del	Endpoint POST /api/v1/users/me/limit-processing que permite al usuario restringir el uso de sus datos.
Registro de auditoría	de	Todas las acciones sensibles (login, exportación de datos, solicitud de borrado) quedan registradas en la tabla audit_log con marca de tiempo, identificador de usuario y hash de la IP.

Desde el punto de vista de la seguridad técnica, las contraseñas se almacenan exclusivamente en su forma hasheada mediante BCrypt y nunca se transmiten ni registran en texto plano. La autenticación se basa en tokens JWT firmados con HMAC-SHA256. En el entorno de producción, toda la comunicación entre la app móvil y el servidor se realiza a través de HTTPS. Durante el desarrollo se permite tráfico HTTP restringido a las IPs del entorno local, configuración que se deshabilita en el build de release mediante las buildConfigField definidas en el build.gradle de la aplicación.

2.3. REQUISITOS DEL SISTEMA

2.3.1. Requisitos funcionales

ID	Descripción	Actor	Prioridad
RF-01	El sistema permitirá el registro de nuevos usuarios con nombre, email, contraseña y tipo de vehículo.	Ambos	Alta
RF-02	El sistema permitirá el inicio de sesión con email y contraseña, devolviendo un token JWT firmado con HMAC-SHA256. La validez del token se configura mediante la propiedad jwt.expiration	Ambos	Alta
RF-03	El sistema permitirá recuperar la contraseña mediante un código de verificación de 6 dígitos con validez de 15 minutos. En el entorno actual el código se registra en el log del servidor (mock); la	Ambos	Mediana

	integración con un servidor de correo real queda como trabajo futuro.		
RF-04	El sistema permitirá crear rutas de reparto con nombre, fecha planificada, notas opcionales y lista de paradas geolocalizadas.	Ambos	Alta
RF-05	El sistema permitirá editar y eliminar rutas existentes. El borrado será lógico, nunca físico.	Ambos	Alta
RF-06	El sistema calculará el orden óptimo de visita de paradas mediante el algoritmo del vecino más cercano (Nearest Neighbor), adecuado para rutas de hasta 20 paradas en un contexto académico y sustituible por algoritmos más complejos sin cambios en la arquitectura.	Ambos	Alta
RF-07	El sistema calculará la distancia total y el tiempo estimado de la ruta consultando la API de OSRM. En caso de fallo de conectividad con OSRM, el cálculo se realizará mediante la fórmula de Haversine como fallback.	Ambos	Alta
RF-08	El sistema permitirá confirmar entregas mediante el escaneo de un código QR o de barras (Code 128) asociado al identificador único de la parada.	Repartidor	Alta
RF-09	El sistema funcionará sin conexión a internet, almacenando los datos localmente mediante Room. Las confirmaciones de entrega realizadas en modo offline se sincronizarán automáticamente con el servidor al recuperar la conexión, mediante WorkManager.	Repartidor	Alta
RF-10	El sistema generará etiquetas logísticas en formato PDF para cada parada, incluyendo datos del destinatario, código de barras Code 128 e identificador QR. La generación se realizará de forma asíncrona para no bloquear la respuesta de la API.	Ambos	Mediana
RF-11	El sistema buscará y sugerirá direcciones mediante un sistema de geocodificación de tres niveles: caché en memoria, base de datos local PostgreSQL y geocodificador externo (OSMDroid/Nominatim) como fallback.	Ambos	Mediana
RF-12	El sistema permitirá exportar todos los datos personales del usuario en formato JSON, en cumplimiento del derecho de portabilidad del RGPD.	Ambos	Alta
RF-13	El sistema permitirá solicitar la eliminación de la propia cuenta. El sistema aplicará borrado lógico inmediato y anonimización automática de los datos personales transcurridos 7 días.	Ambos	Alta

RF-14	El sistema registrará en un log de auditoría las acciones sensibles (inicio de sesión, exportación de datos, solicitud de eliminación de cuenta, cambio de contraseña), incluyendo marca de tiempo, identificador de usuario y hash SHA-256 de la IP.	Sistema	Alta
RF-15	El sistema aplicará rate limiting por IP para prevenir abusos, con límites diferenciados por endpoint: 5 peticiones/minuto en login, 3 en registro y 30 en geocodificación.	Sistema	Alta
RF-16	El sistema enviará notificaciones push al repartidor cuando esté a menos de 500 metros de la siguiente parada pendiente, y cuando se complete la sincronización de entregas realizadas en modo offline, siempre que el usuario haya otorgado su consentimiento explícito.	Repartidor	Baja

2.3.2. Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Rendimiento	Como objetivo de diseño, la API responderá en menos de 500 ms para operaciones de lectura bajo carga normal. La optimización de una ruta de 20 paradas, que combina el algoritmo del vecino más cercano con una consulta a OSRM, no superará los 3 segundos en condiciones normales de red.
RNF-02	Seguridad	Todas las comunicaciones entre la app y la API se realizarán sobre HTTPS en el entorno de producción. Las contraseñas se almacenarán hasheadas con BCrypt (factor de coste por defecto de Spring Security, equivalente a ≥ 10 rondas). Los JWT se firmarán con HMAC-SHA256 usando una clave secreta de 256 bits mínimo, configurada como variable de entorno.
RNF-03	Disponibilidad offline	La app Android funcionará en modo degradado sin conexión: lectura de rutas y paradas desde la base de datos local Room, y confirmación de entregas con sincronización diferida mediante WorkManager al recuperar la conexión. El estado de sincronización de cada parada será visible en la interfaz.
RNF-04	Usabilidad	La app será operable en orientación vertical. Los elementos de acción principal (confirmar entrega, escanear) estarán accesibles en la zona inferior de la pantalla. La interfaz adaptará su disposición al modo horizontal (landscape) sin pérdida de funcionalidad.

RNF-05	Compatibilidad	La app Android será compatible con API level 24 (Android 7.0 Nougat) o superior, tal como se define en el minSdk del build.gradle, cubriendo más del 95% de los dispositivos Android activos en el momento del desarrollo.
RNF-06	Mantenibilidad	El código seguirá los principios SOLID y el patrón Clean Architecture con separación estricta en capas (presentación, dominio, datos).
RNF-07	Privacidad	El sistema cumplirá el RGPD: recogida de datos mínimos necesarios, consentimiento explícito para notificaciones y geolocalización, derechos ARSULIPO implementados mediante endpoints dedicados, y anonimización automática de datos personales transcurridos 7 días desde la solicitud de borrado.
RNF-08	Portabilidad	El backend se podrá desplegar en cualquier sistema operativo con Docker y Docker Compose instalados, sin modificar el código fuente, mediante el archivo docker-compose.yml incluido en el proyecto.
RNF-09	Internacionalización	La app ofrecerá la interfaz en cinco idiomas: español, inglés, francés, portugués y alemán, gestionados mediante el sistema de recursos de Android (strings.xml por locale). El cambio de idioma se aplica mediante AppCompatActivity.setApplicationLocales() y requiere el reinicio de la Activity para su correcta aplicación.
RNF-10	Tematización	La app ofrecerá modo claro y modo oscuro, además de la opción de seguir la configuración del sistema. La preferencia del usuario se persistirá en DataStore y se aplicará en cada arranque de la aplicación.

2.3.3. Restricciones del sistema

ID	Categoría	Descripción
RST-01	Plataforma única móvil	El sistema no implementa soporte nativo para iOS en esta versión. La app está desarrollada exclusivamente para Android (API level 24+). La API REST está diseñada de forma agnóstica a la plataforma cliente, por lo que podría dar soporte a una futura app iOS sin modificaciones en el backend.

RST-02	Algoritmo de optimización	El algoritmo de optimización de rutas implementado es el del vecino más cercano (Nearest Neighbor), que ofrece resultados aceptables para rutas de hasta aproximadamente 20 paradas. Para rutas con un número mayor de paradas el resultado puede alejarse del óptimo global. No existe un límite técnico impuesto en el código; la restricción es de calidad del resultado, no de funcionamiento del sistema.
RST-03	Instancia pública de OSRM	El cálculo de distancias y tiempos reales utiliza la instancia pública de demo de OSRM (router.project-osrm.org). Esta instancia no ofrece garantías de disponibilidad ni de rendimiento bajo carga. El sistema implementa un fallback automático al cálculo por la fórmula de Haversine cuando OSRM no está disponible. En un entorno de producción real se recomienda desplegar una instancia OSRM propia con datos de OpenStreetMap actualizados.
RST-04	Envío de email	El sistema no integra un servidor SMTP real en esta versión. El envío del código de recuperación de contraseña está implementado como mock: el código se genera correctamente y se registra en el log del servidor, pero no se envía al usuario por email. La integración con un proveedor de correo transaccional (SendGrid, Mailgun, AWS SES) queda como trabajo futuro.
RST-05	Acceso a endpoints de etiquetas PDF	El endpoint de generación de etiquetas PDF (GET /api/v1/routes/stops/{stopId}/label) es accesible sin autenticación en la implementación actual, tal como se define en SecurityConfig. La restricción de acceso por rol queda pendiente de implementación en una versión futura.
RST-06	Despliegue en red local	En la versión actual el sistema está diseñado para desplegarse en una red local o privada. El acceso desde la app móvil se realiza mediante la IP de la red local. No se ha configurado un dominio público con certificado SSL propio para producción.

2.3.4. Catalogación y priorización de los requisitos

Los requisitos se han priorizado mediante la técnica MoSCoW (Must, Should, Could, Won't), que permite distinguir entre lo imprescindible para el funcionamiento del sistema y lo deseable dentro del alcance del proyecto.

- Must have — Imprescindibles. Requisitos sin los cuales el sistema no cumple su propósito principal: RF-01, RF-02, RF-04, RF-05, RF-06, RF-07, RF-08, RF-09, RF-12, RF-13, RF-14, RF-15.
- Should have — Importantes. Requisitos de alto valor que complementan el núcleo del sistema pero cuya ausencia no lo invalida: RF-03 (recuperación de contraseña), RF-10 (generación de etiquetas PDF por parada), RF-11 (geocodificación con caché de tres niveles).
- Could have — Deseables. Funcionalidades implementadas o parcialmente implementadas que aportan valor añadido pero no son críticas: RF-16 (notificaciones push de proximidad y sincronización), panel de estadísticas de reparto por repartidor, exportación de un resumen completo de ruta en PDF (distinto de las etiquetas individuales de RF-10).
- Won't have — Fuera de alcance. Funcionalidades expresamente excluidas de esta versión, documentadas para evitar malentendidos sobre el alcance del proyecto: soporte para iOS, integración con sistemas ERP o de facturación, envío real de email (RST-04), panel web de administración multiempresa, restricción de endpoints por rol (RST-05).

2.4. ALTERNATIVAS Y SELECCIÓN DE LA SOLUCIÓN

Para cubrir los requisitos identificados se analizaron tres alternativas tecnológicas para el componente cliente. El backend REST con Spring Boot se consideró la elección natural desde el inicio dado el currículum del ciclo formativo, y no fue objeto de comparativa formal.

2.4.1. Alternativa 1: Aplicación web progresiva (PWA)

Una Progressive Web App permitiría acceder al sistema desde cualquier navegador móvil sin instalación previa mediante una URL. Se implementaría con un framework JavaScript moderno (React, Vue o Angular) combinado con Service Workers para el modo offline.

Ventajas:

- Sin instalación: el usuario accede directamente desde el navegador.
- Multiplataforma: funciona en Android, iOS y escritorio con el mismo código.
- Despliegue simple: se sirve como archivos estáticos desde el backend.

Inconvenientes:

- El acceso a la cámara mediante la API de MediaDevices para el escaneo de códigos funciona en algunos navegadores, pero con experiencia inferior a una app nativa: sin enfoque automático configurable ni vibración de confirmación.
- Las notificaciones push en iOS tienen soporte limitado y requieren que el usuario añada la PWA a la pantalla de inicio.

- La sincronización diferida en segundo plano mediante Service Workers es considerablemente menos fiable que WorkManager nativo en Android, especialmente en condiciones de batería baja.
- El renderizado de mapas con librerías JavaScript es significativamente más pesado que OSMDroid nativo en dispositivos de gama media-baja, que son los más comunes en el sector logístico.
- El alumno tiene mayor dominio del desarrollo Android que del desarrollo web frontend, lo que supondría un coste de arranque elevado en el plazo disponible.

Conclusión: descartada por las limitaciones en el acceso a la cámara, la menor fiabilidad del modo offline y el rendimiento en mapas.

2.4.2. Alternativa 2: Aplicación multiplataforma con Flutter

Flutter es el framework de Google para el desarrollo de aplicaciones nativas multiplataforma (Android, iOS, Web, Desktop) con un único código fuente en el lenguaje Dart.

Ventajas:

- Un único código fuente para Android e iOS, lo que maximizaría el alcance del producto.
- Rendimiento nativo gracias a su motor de renderizado propio (Skia/Impeller), superior al de React Native.
- Ecosistema de paquetes maduro en pub.dev, con equivalentes para Room (Drift), mapas (flutter_map) y escaneo de códigos.

Inconvenientes:

- Curva de aprendizaje de Dart: el alumno no tiene experiencia previa con este lenguaje, lo que supondría un coste de formación significativo dentro del plazo del proyecto.
- Flutter no expone directamente WorkManager de Android. Las tareas de sincronización en segundo plano requieren plugins de terceros con menor garantía de mantenimiento y comportamiento menos predecible que la integración nativa, siendo WorkManager un componente crítico para la arquitectura offline-first del sistema.
- Para el perfil de usuario objetivo (repartidores en España), iOS tiene una cuota de mercado aproximada del 25%, insuficiente para justificar el sobrecoste de aprendizaje en el plazo disponible.

Conclusión: descartada principalmente por el coste de aprendizaje de Dart y la integración no nativa con WorkManager.

2.4.3. Alternativa 3: Aplicación nativa Android con Kotlin — SELECCIONADA

El desarrollo de una aplicación Android nativa con Kotlin y Jetpack Compose es la alternativa seleccionada por las siguientes razones:

- Kotlin es el lenguaje oficial de Android, trabajado durante el módulo de Programación Multimedia y Dispositivos Móviles del ciclo

formativo, lo que reduce significativamente el coste de arranque del proyecto.

- Jetpack Compose es el toolkit de UI moderno recomendado por Google para Android: reactivo, declarativo y perfectamente integrado con el patrón MVVM y Kotlin Flow.
- WorkManager ofrece las garantías más robustas del ecosistema Android para la ejecución de tareas en segundo plano con políticas de reintento y restricciones de red, siendo la solución nativa para la arquitectura offline-first requerida.
- Room es la solución oficial de persistencia local en Android, con soporte completo para Kotlin Coroutines y Flow, y es el componente central de la estrategia offline-first.
- El perfil de usuario objetivo usa mayoritariamente Android en España.
- El backend desacoplado mediante API REST permite añadir un cliente iOS en el futuro sin modificar la lógica de negocio.

2.4.4. Tabla comparativa de alternativas

Criterio	PWA	Flutter	Android Nativo
Rendimiento offline	Medio	Alto	Alto
Acceso a cámara nativo	Limitado	Alto	Alto
Conocimiento previo del alumno	Bajo	Bajo	Alto
Integración WorkManager	No Nativa	Plugin de terceros	Nativa
Rendimiento en mapas	Medio	Alto	Alto
Soporte iOS futuro	Sí	Sí	No sin nuevo proyecto
Seleccionada	No	No	Sí

2.5. PLANIFICACIÓN DEL PROYECTO

2.5.1. Metodología de desarrollo

El proyecto se ha desarrollado siguiendo una metodología iterativa e incremental inspirada en Scrum. Dado que el equipo es de una sola persona, se simplificaron las ceremonias formales del framework: se realizaron revisiones semanales del avance asumidas por el propio alumno, y reuniones periódicas con la tutora del proyecto para seguimiento del avance y validación de las decisiones técnicas adoptadas.

El trabajo se dividió en ocho fases de duración variable, cada una con un conjunto definido de funcionalidades a implementar. Al final de cada fase el

sistema quedaba en un estado funcional y desplegable, lo que permitía validar el progreso de forma continua y detectar desviaciones antes de comenzar la siguiente fase.

El control de versiones se gestionó mediante Git con repositorio en GitHub, con commits descriptivos que documentan la evolución del proyecto a lo largo de las ocho fases. Esto permite trazar la historia del desarrollo y justificar las decisiones técnicas adoptadas en cada momento.

2.5.2. Recursos del proyecto

Recursos humanos: un desarrollador a tiempo parcial durante los meses de enero a mayo de 2026, con dedicación variable en función de la carga lectiva del ciclo formativo, totalizando 368 horas de trabajo efectivo registradas a lo largo del proyecto.

Recursos materiales:

- Ordenador portátil de desarrollo: procesador AMD Ryzen 9 5900HX (16 núcleos), 32 GB RAM, SSD 1 TB + SSD 500GB, sistema operativo Windows 10.
- Dispositivo Android físico para pruebas: Xiaomi Redmi Note 9 PRO, Android 12 (API 31), 4 GB RAM.
- Emulador Android: Medium Phone API 31 integrado en Android Studio.

Recursos software (todos de código abierto o gratuitos para uso educativo):

- Android Studio Narwhal Feature Drop 2025.1.3 — IDE principal para el desarrollo de la app Android con Kotlin y Jetpack Compose.
- IntelliJ IDEA Community Edition 2025.2.1 — IDE para el desarrollo del backend Java/Spring Boot.
- Docker Desktop 4.x (Windows) — orquestación de contenedores para el despliegue local del backend y la base de datos.
- PostgreSQL 14 (contenedor Docker) — base de datos relacional del backend.
- Git 2.43 + GitHub — control de versiones y repositorio remoto del proyecto.
- Postman / IntelliJ HTTP Client — pruebas manuales de los endpoints de la API REST.
- Figma — diseño del prototipo de interfaz de usuario previo al desarrollo. El prototipo recoge las pantallas principales de la app con el sistema de diseño Neo-Brutalista definido para el proyecto.
- draw.io — elaboración de diagramas UML y de arquitectura.
- Google Docs — redacción de la memoria del proyecto.

2.5.3. División del trabajo en fases

Fase	Descripción	Inicio prev.	Fin prev.	Horas totales
------	-------------	--------------	-----------	---------------

F0	Planificación, selección de tecnologías, análisis de requisitos y configuración del entorno de desarrollo.	Ene 2026	Ene 2026	18h
F1	Cimientos del backend: estructura de proyectos, entidades JPA, migraciones Flyway, configuración Docker Compose y esquema de base de datos.	Ene 2026	Feb 2026	22h
F2	Autenticación y seguridad: JWT con HMAC-SHA256, Spring Security, endpoints de registro, login, recuperación y cambio de contraseña.	Feb 2026	Feb 2026	24h
F3	Motor de rutas backend: algoritmo del vecino más cercano, integración OSRM, fallback Haversine, endpoints REST de rutas y paradas.	Mar 2026	Mar 2026	30h
F4	Geocodificación híbrida: caché en memoria, caché persistente en PostgreSQL con JSONB, búsqueda en base de datos local con índices trigram.	Mar 2026	Abr 2026	18h
F5	Cumplimiento RGPD:	Abr 2026	Abr 2026	22h

	exportación JSON, borrado lógico, anonimización automática con @Scheduled, registro de auditoría con hash SHA-256 de IP.			
F6	App Android — núcleo: arquitectura MVVM, inyección de dependencias con Hilt, cliente Retrofit, Room, pantallas de autenticación en Jetpack Compose y navegación con Compose Navigation.	Abr 2026	May 2026	38h
F7	App Android — rutas y mapas: pantallas de listado, detalle y creación de rutas, integración OSMDroid, Place Picker, escaneo de códigos con ZXing.	May 2026	May 2026	28h
F8	Offline-first: Room como fuente de verdad, WorkManager para sincronización diferida con reintentos exponenciales, monitor de conectividad y UI de estado de sincronización.	May 2026	May 2026	22h
F9	Etiquetas PDF con OpenPDF y ZXing, rate limiting con Bucket4j, modo oscuro/claro, internacionalizació	May 2026	May 2026	22h

	n en 5 idiomas, ajustes finales y pruebas.			
Redacción memoria		Ene 2026	May 2026	78h
Pruebas		Abr 2026	May 2026	18h
Planificación /Bibliografía				28h
Total		368 h		

2.5.4. Estimación de tiempos

Actividad	Días estimados	Horas estimadas	Horas reales
Planificación y bibliografía	4	50h	46h
Fase 1: Cimientos backend	5	25h	22h
Fase 2: Autenticación y seguridad	4	20h	24h
Fase 3: Motor de optimización	5	25h	30h
Fase 4: Geocodificación híbrida	4	20h	18h
Fase 5: RGPD y auditoría	4	20h	22h
Fase 6: App Android (UI + mapas)	6	30h	38h
Fase 7: Offline-first y WorkManager	5	25h	28h
Fase 8: PDF, rate limiting, i18n, temas	4	20h	22h
Pruebas integrales	4	20h	18h
Redacción de la memoria	14	70h	78h
TOTAL	69 días	345h	368h

La desviación total fue de +23 horas sobre la estimación inicial de 345 horas (+6,7%). Entre las principales causas destacan la mayor complejidad de la integración de OSMDroid con Jetpack Compose, un problema de incompatibilidad de la librería OpenPDF con Java 17, y la extensión de la redacción de la memoria respecto a lo previsto. Todas las funcionalidades planificadas se completaron dentro del plazo final.

2.5.5. Planificación temporal

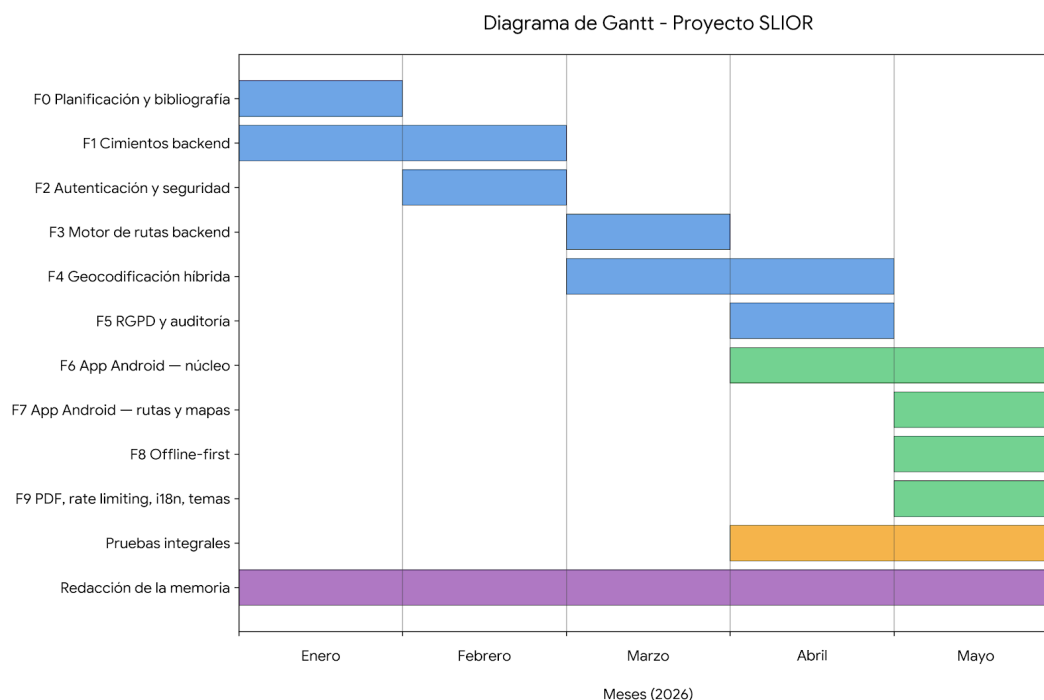


Figura 2.1. Diagrama de Gantt del proyecto **SLIOR**.

2.6. EVALUACIÓN DE RIESGOS

2.6.1. Identificación de riesgos

La exposición se calcula como el producto cualitativo de la probabilidad por el impacto: Alta = probabilidad Media o Alta con impacto Alto o Muy Alto; Media = combinaciones intermedias; Baja = probabilidad Baja con impacto Bajo o Medio.

ID	Descripción del riesgo	Prob.	Impacto	Exposición
R-01	Incompatibilidad de versiones entre librerías del ecosistema Android (Compose, Hilt, Room, WorkManager).	Media	Alto	Alta
R-02	Servicio público de OSRM no disponible durante el desarrollo o pruebas.	Baja	Medio	Media
R-03	Desviación temporal significativa por la complejidad de la capa offline.	Media	Alto	Alta
R-04	Cambio de comportamiento de APIs de Android relacionadas	Baja	Medio	Media

	con permisos de cámara en versiones nuevas.			
R-05	Fallo del hardware de desarrollo (disco duro, etc.) con pérdida de trabajo.	Baja	Muy alto	Media
R-06	Cambio en los términos de uso o disponibilidad de los tiles de OpenStreetMap.	Baja	Baja	Baja

2.6.2. Plan de contingencia

- R-01: uso del parent POM de Spring Boot para la gestión centralizada de versiones de dependencias, y del BOM de Jetpack Compose en el proyecto Android. Ante cualquier problema de compilación, verificar la tabla de compatibilidad de Compose en la documentación oficial.
- R-02: implementar el fallback a la fórmula Haversine desde el principio, no como medida de emergencia. Probar siempre con el fallback activado para garantizar su correcto funcionamiento.
- R-03: identificar las funcionalidades de menor valor (notificaciones push, estadísticas) como candidatas a recorte si el tiempo se ajusta. Proteger el núcleo (RF-08, RF-09) a cualquier coste.
- R-04: leer las notas de versión de Android al actualizar compileSdkVersion. Usar ActivityResultContracts para la gestión de permisos, que es la API recomendada y más estable.
- R-05: uso de un repositorio Git privado en GitHub con commits frecuentes (al menos una vez al día). Copia de seguridad semanal en disco externo.
- R-06: OSMDroid almacena las teselas descargadas en caché local, por lo que el mapa sigue funcionando offline con los datos ya cargados. En caso de cambio en los términos de uso, OpenStreetMap dispone de servidores de tiles alternativos compatibles con OSMDroid configurables sin cambios en el código.

2.7. PRESUPUESTO

2.7.1. Estimación de coste material

Concepto	Coste total	Amortización	Imputable
Ordenador de desarrollo (valor: 1.400€, vida útil 4 años, uso 5 meses)	1.400 €	48 meses	145,83 €
Dispositivo Android para pruebas (valor: 280€, vida útil 3 años, uso 5 meses)	280 €	36 meses	38,89 €

Monitor externo (valor: 200€, vida útil 5 años, uso 5 meses)	200 €	60 meses	16,67 €
Conexión a internet (20€/mes × 33% uso profesional × 5 meses)	—	—	33,33 €
Software (todo open source o licencias educativas gratuitas)	0 €	—	0,00 €
Servidor de despliegue (red local, sin coste adicional)	0 €	—	0,00 €
TOTAL COSTE MATERIAL			234,72 €

2.7.2. Estimación de coste personal

La distribución de horas por rol se ha obtenido a partir del registro de actividad por fases recogido en la tabla 2.5.4, asignando cada fase al perfil profesional que correspondería en un equipo real.

Perfil / Rol	Horas	€/hora	Subtotal	Descripción
Jefe de proyecto / Analista	40	30 €	1.200 €	Planificación, análisis de requisitos, gestión de riesgos.
Arquitecto de software	20	35 €	700 €	Diseño de arquitectura, selección tecnológica.
Desarrollador Backend (Java/Spring)	100	28 €	2.800 €	Implementación API REST, seguridad, RGPD, Docker.
Desarrollador Android (Kotlin)	110	28 €	3.080 €	Implementación app Android, UI Compose, offline.
Tester / QA	30	22 €	660 €	Plan de pruebas, ejecución de pruebas, informe.
Técnico de documentación	68	20 €	1.360 €	Redacción de la

				memoria y glosario.
TOTAL COSTE PERSONAL	368		9.800 €	

Nota: todos los roles del proyecto han sido desempeñados por el mismo alumno. El coste personal refleja el valor de mercado que tendría el trabajo realizado en un entorno profesional.

2.7.3. Resumen y análisis coste-beneficio

Concepto	Importe
Coste material	234,72 €
Coste personal	9.800,00 €
COSTE TOTAL DEL PROYECTO	10.034,72 €
Ahorro anual en licencias (vs. OptimoRoute Pro, 1 conductor)	~600 €/año
Ahorro anual en combustible (gasto medio 4.000€/año, reducción estimada del 10% por optimización de rutas)	~400 €/año
Ahorro en tiempo de planificación (1h/día × 5 días × 48 semanas × 20€/h)	~4.800 €/año
Beneficio total estimado anual	~5.800 €/año
Retorno de inversión (ROI) estimado	~21 meses

2.8. CONCLUSIONES DEL ESTUDIO DE VIABILIDAD

El proyecto **SLIOR** es técnica, económicamente y normativamente viable. La tecnología seleccionada (Spring Boot + Kotlin + Android nativo) está madura y bien documentada, y el alumno dispone de los conocimientos necesarios para abordarla en el plazo disponible.

El coste total del proyecto (10.034,72 €) se amortiza en aproximadamente 21 meses mediante el ahorro en licencias, combustible y tiempo de planificación para un repartidor autónomo. El uso exclusivo de software de código abierto garantiza la sostenibilidad económica del sistema a largo plazo, eliminando los costes recurrentes de licencias y la dependencia de proveedores externos.

El principal riesgo identificado (desviación temporal en la capa offline) se ha materializado de forma leve (+3 horas respecto a lo estimado) sin afectar al plazo final. La implementación del fallback a Haversine ha mitigado completamente el riesgo de indisponibilidad de OSRM.

3. ANÁLISIS

3.1. INTRODUCCIÓN

En este capítulo se describe el comportamiento esperado del sistema **SLIOR** mediante un catálogo de actores, historias de usuario, casos de uso, el modelo de datos y los diagramas de interacción para los flujos principales.

3.2. CATÁLOGO DE ACTORES

Se identifican dos actores en el sistema. El enum UserRole define los valores REPARTIDOR y ADMINISTRADOR en el modelo de datos para soportar la distinción futura, pero en la versión actual ambos perfiles tienen acceso idéntico a todas las funcionalidades del sistema.

Actor	Descripción	Rol en el sistema
Usuario autenticado	Repartidor o gestor de flota registrado en el sistema. En esta versión ambos perfiles acceden a las mismas funcionalidades a través de la app móvil.	Gestiona rutas y paradas, confirma entregas, trabaja en modo offline. La diferenciación de permisos por rol queda como trabajo futuro (RST-05).
Sistema (Backend)	Actor no humano. Ejecuta tareas automáticas programadas.	Anonimización automática de datos (RGPD), auditoría de acciones sensibles, generación asíncrona de etiquetas PDF.

3.3. HISTORIAS DE USUARIO

Las historias de usuario se han elaborado siguiendo el formato estándar: "Como [rol], quiero [acción], para [beneficio]". Cada historia incluye sus criterios de aceptación, que definen cuándo la historia se considera implementada correctamente.

HU-01 — Registro de usuario

Como:	Nuevo usuario (repartidor o administrador)
Quiero:	Registrarme en el sistema con mi nombre, email, contraseña y tipo de vehículo
Para:	Poder acceder a mis rutas de trabajo desde la app Android

Criterios de aceptación:	1. El email debe ser único en el sistema. Si ya existe, mostrar error descriptivo. 2. La contraseña debe tener al menos 8 caracteres, una mayúscula y un número. Esta validación se aplica en el cliente Android; el backend valida únicamente la longitud mínima de 8 caracteres. 3. El tipo de vehículo debe ser CAR, VAN o TRUCK. 4. Al registrarse correctamente, el sistema devuelve un JWT válido y la app navega a la pantalla principal. 5. La contraseña nunca se almacena en texto claro.
Prioridad:	Alta

HU-02 — Inicio de sesión

Como:	Usuario registrado
Quiero:	Iniciar sesión con mi email y contraseña
Para:	Acceder a mis rutas y gestionar mis entregas
Criterios de aceptación:	1. Con credenciales correctas, el sistema devuelve un JWT con 24 horas de validez. 2. Con credenciales incorrectas, el sistema devuelve HTTP 401 con mensaje de error sin revelar si el email existe o no (protección anti-enumeración). 3. Tras 5 intentos fallidos en 1 minuto desde la misma IP, el sistema devuelve HTTP 429 (rate limit). 4. El JWT se almacena en DataStore. El cifrado mediante Android Keystore queda como mejora para una versión futura.
Prioridad:	Alta

HU-03 — Consultar mis rutas del día

Como:	Repartidor
Quiero:	Ver la lista de mis rutas asignadas para hoy y los próximos días
Para:	Saber cuántas entregas tengo y en qué estado están
Criterios de aceptación:	1. La pantalla principal muestra las rutas ordenadas por fecha planificada. 2. Cada tarjeta de ruta muestra: nombre, fecha, número de paradas, número entregas y estado general. 3. La pantalla se carga correctamente aunque no haya conexión (datos desde Room). 4. Un indicador visual (icono de nube) muestra si los datos están sincronizados con el servidor.
Prioridad:	Alta

HU-04 — Optimizar el orden de entregas

Como:	Repartidor o Administrador
Quiero:	Que el sistema calcule el orden más eficiente de visita de las paradas pendientes
Para:	Reducir los kilómetros totales recorridos y el tiempo de jornada
Criterios de aceptación:	1. El botón "Optimizar" está disponible en la pantalla de detalle de ruta. 2. El sistema optimiza solo las paradas en estado PENDIENTE; las ya entregadas mantienen su posición. 3. El resultado muestra la distancia total en km y el tiempo estimado en minutos. 4. Si OSRM no está disponible, el sistema aplica automáticamente el fallback mediante la fórmula Haversine como estimación de distancia y tiempo. 5. El nuevo orden se guarda automáticamente.
Prioridad:	Alta

HU-05 — Confirmar entrega offline

Como:	Repartidor
Quiero:	Confirmar una entrega escaneando el código del paquete, aunque no tenga cobertura en ese momento
Para:	Registrar el evento con la hora exacta y que el sistema lo sincronice automáticamente después
Criterios de aceptación:	1. La pantalla de escaneo funciona sin conexión. 2. La confirmación se registra en Room inmediatamente con marca temporal. 3. La parada cambia visualmente a estado ENTREGADO al instante. 4. Un icono de nube con flecha indica que la confirmación está pendiente de sincronización. 5. Al recuperar la conexión (WiFi o datos), WorkManager sincroniza automáticamente en segundo plano. 6. Tras la sincronización, el icono de nube desaparece.
Prioridad:	Alta

HU-06 — Navegar a la siguiente parada

Como:	Repartidor
Quiero:	Abrir Google Maps (o Waze) con la dirección de la siguiente parada en un solo tap
Para:	Navegar sin tener que copiar y pegar la dirección manualmente

Criterios de aceptación:	1. En la pantalla de parada hay un botón "Navegar" visible y accesible. 2. Al pulsarlo, se abre Google Maps (si está instalado) o el navegador del sistema con la URL de Google Maps con las coordenadas de la parada. 3. Todas las paradas tienen coordenadas obligatorias, garantizadas por validación en el momento de creación, por lo que la navegación siempre dispone de coordenadas precisas.
Prioridad:	Alta

HU-07 — Exportar mis datos personales (RGPD)

Como:	Cualquier usuario autenticado
Quiero:	Descargar todos mis datos personales almacenados en el sistema en formato JSON
Para:	Ejercer mi derecho de portabilidad y acceso establecido en el RGPD
Criterios de aceptación:	1. El endpoint <code>/api/v1/users/me/data-export</code> devuelve un JSON con: datos del perfil, lista de rutas con sus paradas.. 2. La acción queda registrada en el log de auditoría.
Prioridad:	Alta

HU-08 — Cambiar el idioma de la aplicación

Como:	Repartidor
Quiero:	Cambiar el idioma de la app a mi idioma materno
Para:	Usar la app cómodamente si no hablo español
Criterios de aceptación:	1. En la pantalla de configuración hay un selector de idioma con las cinco opciones (ES, EN, FR, PT, DE). 2. El cambio se aplica tras el reinicio automático de la Activity, que el sistema ejecuta de forma transparente para el usuario. 3. La preferencia se persiste entre sesiones.
Prioridad:	Media

3.4. REQUISITOS FUNCIONALES DE USUARIOS

La tabla siguiente resume las funcionalidades disponibles para el actor Usuario autenticado, derivadas del análisis de las historias de usuario. En la versión actual, ambos perfiles (REPARTIDOR y ADMINISTRADOR) tienen acceso idéntico a todas las funcionalidades del sistema, tal como se documenta en RST-05.

Funcionalidad	Usuario Autenticado
Registro e inicio de sesión	Sí
Recuperación de contraseña y Cambio de contraseña	Sí
Ver rutas propias	Sí
Crear / Editar / Eliminar rutas	Sí
Optimizar orden de paradas	Sí
Confirmar entrega (escaneo QR/código de barras)	Sí
Ver mapa de la ruta	Sí
Confirmar entrega (escaneo QR/código)	Sí
Ver mapa de la ruta	Sí
Navegar a parada (Google Maps / Waze)	Sí
Usar modo offline con sincronización diferida	Sí
Generar etiquetas logísticas en PDF	Sí
Buscar y seleccionar dirección (geocodificación)	Sí
Exportar datos propios en JSON (RGPD)	Sí
Solicitar eliminación de cuenta	Sí
Cambiar idioma de la app	Sí
Cambiar tema visual (claro/oscurito)	Sí

3.5. REQUISITOS NO FUNCIONALES — RESTRICCIONES DE DISEÑO

Además de los requisitos no funcionales del capítulo 2.3.2, se añaden las siguientes restricciones derivadas del análisis detallado:

- La API no almacenará estado de sesión en el servidor (stateless JWT). Todos los tokens de sesión se gestionan exclusivamente en el cliente.
- La app Android usará Room como única fuente de verdad para la UI, siguiendo el patrón Single Source of Truth. La interfaz nunca accede directamente a la red: siempre lee de Room y escribe en Room, que se sincroniza en segundo plano.
- Los identificadores de todas las entidades usarán UUID v4, generados por el servidor en el momento de la persistencia, lo que facilita la sincronización distribuida offline sin conflictos de ID y permite referenciar entidades antes de que se complete la sincronización.

- El borrado de datos es siempre lógico en la primera instancia (campo `is_deleted` = true), con anonimización física de los datos personales a los 7 días mediante una tarea programada. Esto garantiza la integridad referencial y el registro de auditoría.
- Todas las peticiones a la API incluirán una cabecera X-Request-ID con un UUID único para la trazabilidad de incidencias en los logs.
- El sistema aplicará rate limiting por IP en los endpoints sensibles mediante Bucket4j, con límites diferenciados por tipo de endpoint y limpieza automática de estado inactivo para evitar fugas de memoria.

3.6. DIAGRAMAS DE CASOS DE USO

3.6.1. Diagrama de casos de uso — Autenticación

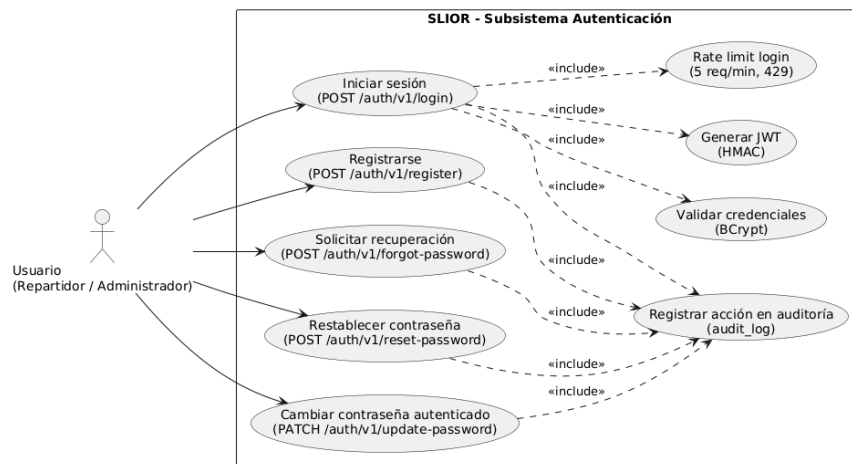


Figura 3.1. Diagrama de casos de uso del subsistema de autenticación.

3.6.2. Diagrama de casos de uso — Gestión de rutas

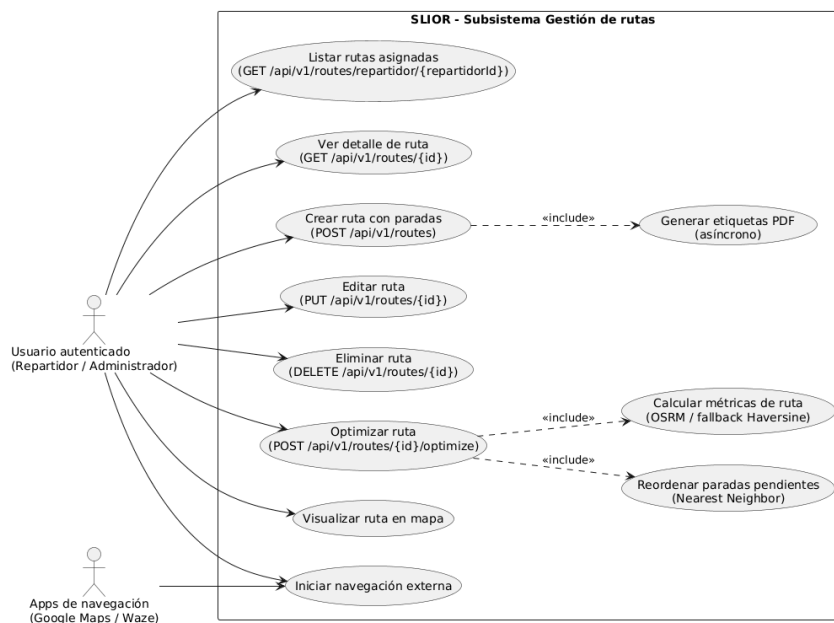


Figura 3.2. Diagrama de casos de uso del subsistema de gestión de rutas.

3.6.3. Diagrama de casos de uso — Gestión de entregas

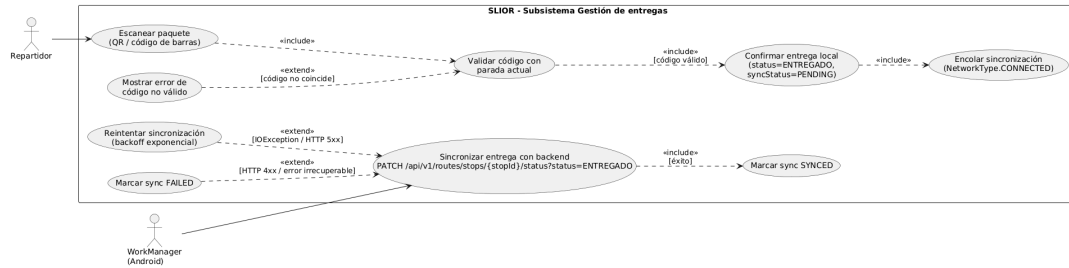


Figura 3.3. Diagrama de casos de uso del subsistema de gestión de entregas.

3.6.4. Diagrama de casos de uso — RGPD y privacidad

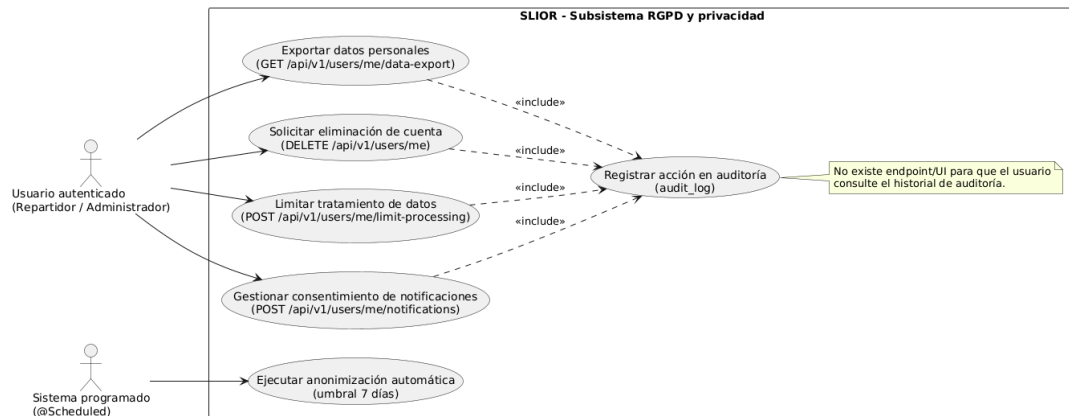


Figura 3.4. Diagrama de casos de uso del subsistema de privacidad y RGPD.

3.6.5. Especificación de casos de uso principales

CU-01 — Iniciar sesión

ID:	CU-01
Nombre:	Iniciar sesión
Actor principal:	Repartidor / Administrador
Precondiciones:	Usuario registrado y no eliminado lógicamente.
Flujo principal:	1. El usuario introduce email y contraseña. 2. La app valida formato antes de enviar. 3. La app envía POST /auth/v1/login. 4. El backend valida credenciales con BCrypt. 5. El backend genera y devuelve JWT firmado (HMAC). 6. La app guarda el token en DataStore y lo usa en Authorization: Bearer ... en peticiones posteriores.

Flujo alternativo A (credenciales incorrectas):	En el paso 4, el servidor devuelve HTTP 401. La app muestra el mensaje "Credenciales incorrectas" sin revelar si el email existe.
Flujo alternativo B (rate limit):	Tras 5 intentos fallidos en 1 minuto, el servidor devuelve HTTP 429. La app muestra "Demasiados intentos. Espera 1 minuto."
Postcondiciones:	Sesión autenticada con JWT persistido en DataStore (sin cifrado adicional explícito en código).
Comprobable en:	AuthController.java AuthService.java JwtUtil.java, RateLimitingFilter.java AuthRepository.kt AuthInterceptor.kt

CU-02 — Optimizar ruta

ID:	CU-02
Nombre:	Optimizar ruta
Actor principal:	Usuario autenticado
Precondiciones:	Existe una ruta con al menos 2 paradas en estado PENDIENTE.
Flujo principal:	1. El usuario pulsa "Optimizar". 2. La app obtiene ubicación actual y envía POST /api/v1/routes/{id}/optimize con puntoInicioLat, puntoInicioLon, vehicleType. 3. El backend aplica Nearest Neighbor sobre pendientes y actualiza ordenVisita. 4. El backend calcula distancia/tiempo con OSRM (si disponible). 5. La app actualiza lista y mapa con el nuevo orden.
Flujo alternativo A (OSRM no disponible):	En el paso 6, la petición a OSRM falla. El servidor aplica la fórmula Haversine como fallback.
Postcondiciones:	Las paradas quedan re-ordenadas. Se actualiza la distancia total y el tiempo estimado de la ruta.
Comprobable en:	RouteController.java RouteOptimizationService.java RouteViewModel.kt RouteRepository.kt ApiService.kt

CU-03 — Confirmar entrega offline

ID:	CU-03
-----	-------

Nombre:	Confirmar entrega (modo offline)
Actor principal:	Repartidor
Precondiciones:	Parada pendiente; puede no haber conexión.
Flujo principal:	1. El repartidor abre detalle de parada y entra en escaneo. 2. La app escanea QR/código de barras con ZXing. 3. La app valida el código frente a la parada actual. 4. Si es válido, marca localmente ENTREGADO, guarda timestamp y syncStatus=PENDING. 5. La app encola SyncDeliveryWorker con NetworkType.CONNECTED. 6. Al recuperar conexión, el worker envía PATCH /api/v1/routes/stops/{stopId}/status?status=ENTREGADO. 7. Si éxito, syncStatus=SYNCED; si error recuperable, reintento; si no recuperable, syncStatus=FAILED.
Flujo alternativo A (código no coincide):	Error en UI y no confirma entrega
Postcondiciones:	Entrega confirmada localmente al instante y sincronizada en diferido con backend.
Comprobable en:	ScanScreen.kt, RouteViewModel.kt, RouteRepository.kt, SyncDeliveryWorker.kt, RouteController.java.

CU-04 — Crear ruta con paradas

ID:	CU-04
Nombre:	Crear ruta con paradas
Actor principal:	Usuario autenticado
Precondiciones:	Usuario autenticado y datos obligatorios válidos.
Flujo principal:	1. El usuario abre formulario de creación. 2. Introduce nombre, fecha y paradas. 3. Selecciona dirección con buscador/mapa para obtener coordenadas. 4. La app envía POST /api/v1/routes. 5. El backend persiste ruta y paradas con estado PLANIFICADA. 6. El backend lanza generación asíncrona de etiquetas PDF.
Flujo alternativo A: Validación fallida de campos obligatorios (incluyendo coordenadas)	No se crea la ruta.

Postcondiciones:	Ruta y paradas persistidas; generación de etiquetas en segundo plano.
Comprobable en:	CreateRouteScreen.kt CreateRouteRequest.java StopRequest.java RouteService.java, LabelService.java SecurityConfig.java

3.7. MODELO LÓGICO DE DATOS

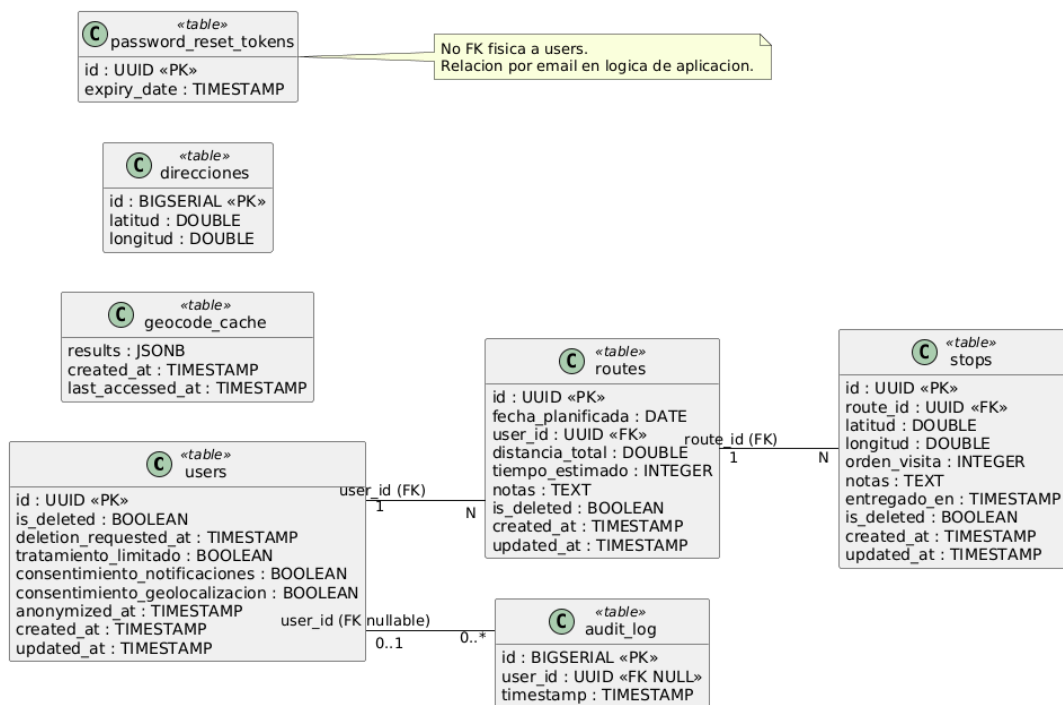


Figura 3.5. Diagrama Entidad-Relación del sistema **SLIOR**.

Las entidades del modelo de datos y sus relaciones son:

- users (1) -> (N) routes mediante routes.user_id (FK NOT NULL a users.id).
- routes (1) -> (N) stops mediante stops.route_id (FK NOT NULL a routes.id).
- users (1) -> (N) audit_log mediante audit_log.user_id (FK nullable a users.id).
- password_reset_tokens no tiene FK a users; la vinculación es por email (integridad referencial no forzada porBD).
- geocode_cache y direcciones son tablas de soporte de geocodificación (sin FK entre ellas en el esquema actual).

La tabla geocode_cache almacena los resultados de geocodificación en una columna de tipo JSONB en PostgreSQL.

3.8. DIAGRAMA DE CLASES DEL BACKEND

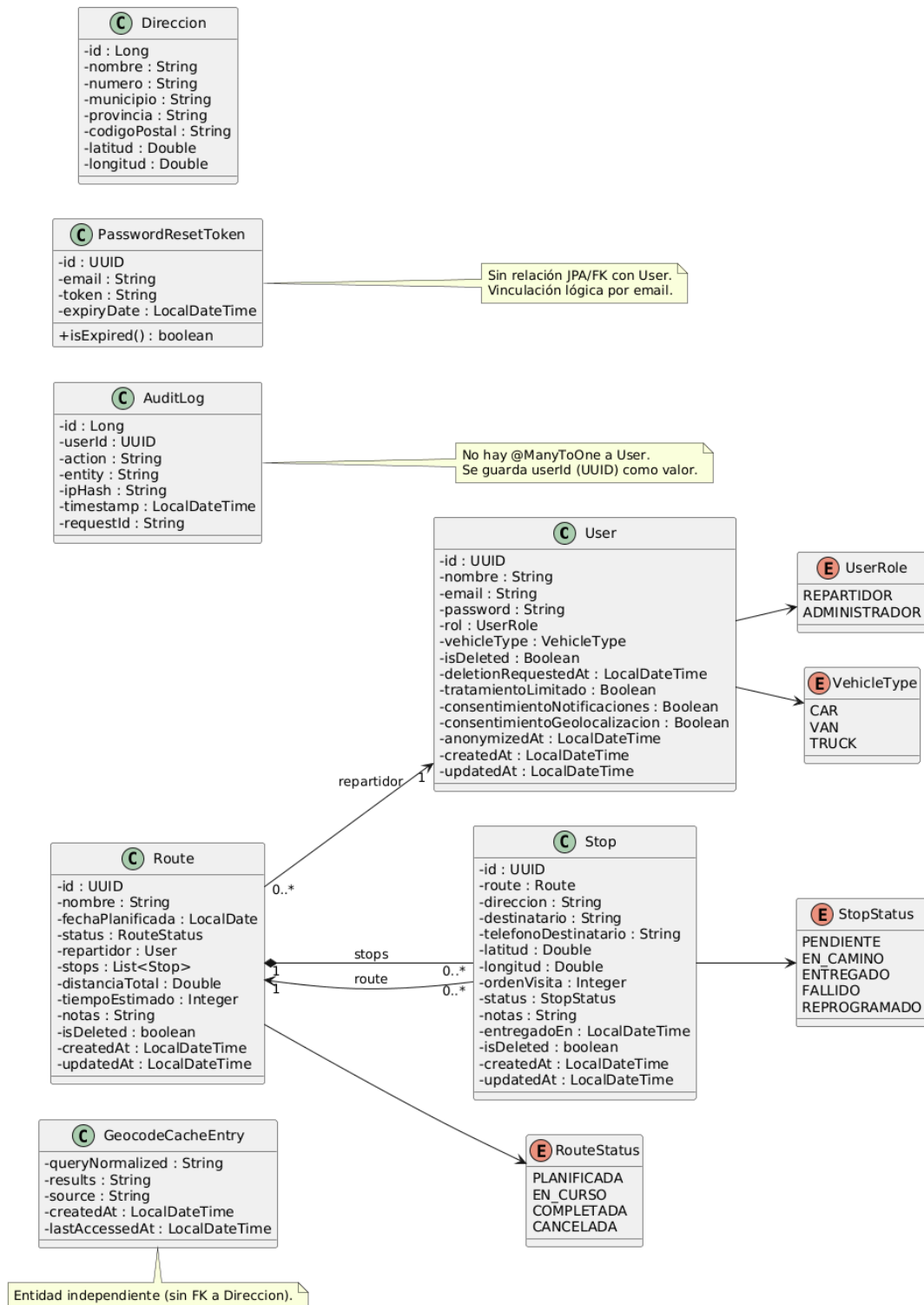


Figura 3.6. Diagrama de clases de las entidades del backend.

3.9. DIAGRAMA DE CLASES DE LA APLICACIÓN ANDROID

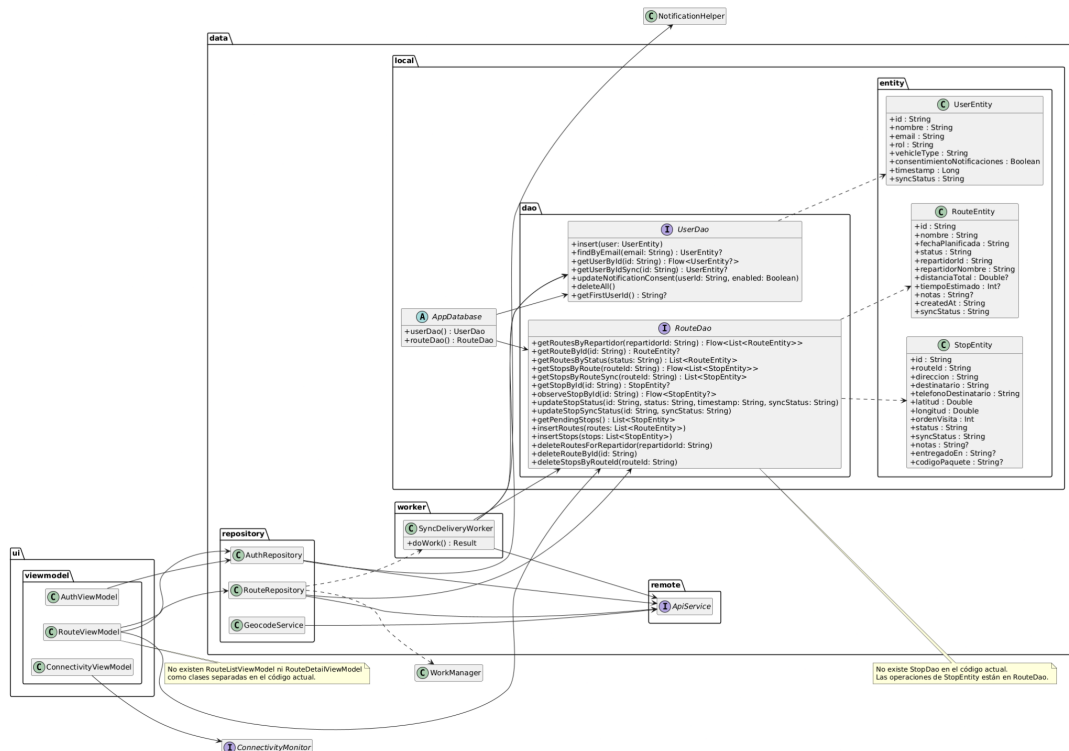


Figura 3.7. Diagrama de clases de la aplicación Android.

3.10. DIAGRAMAS DE INTERACCIÓN

3.10.1. Diagrama de secuencia — Flujo de inicio de sesión

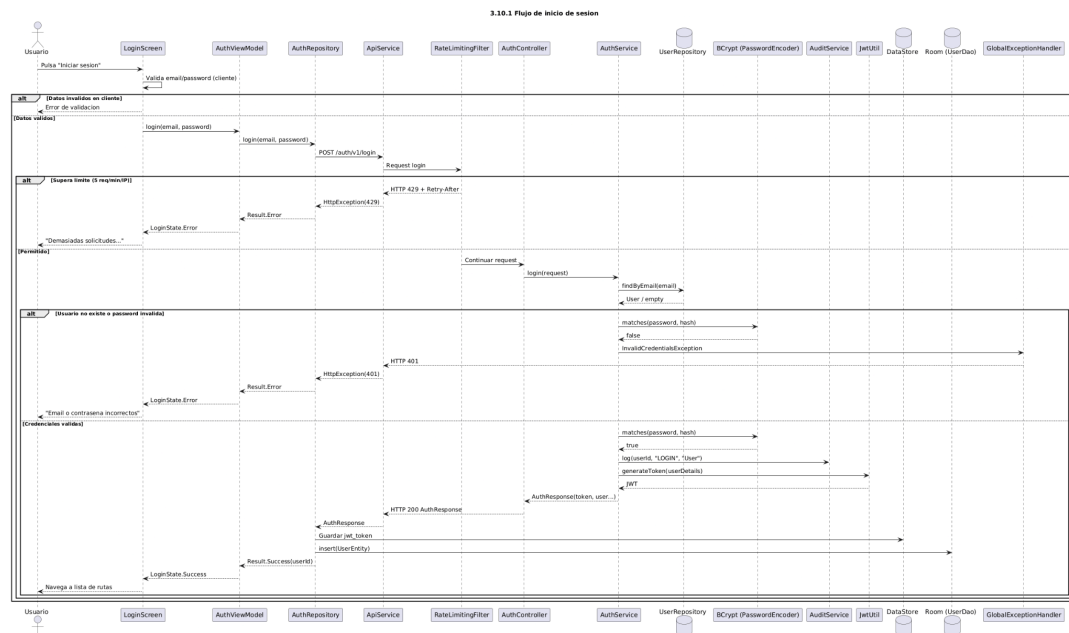


Figura 3.8. Diagrama de secuencia del flujo de inicio de sesión.

3.10.2. Diagrama de secuencia — Flujo de confirmación offline y sincronización

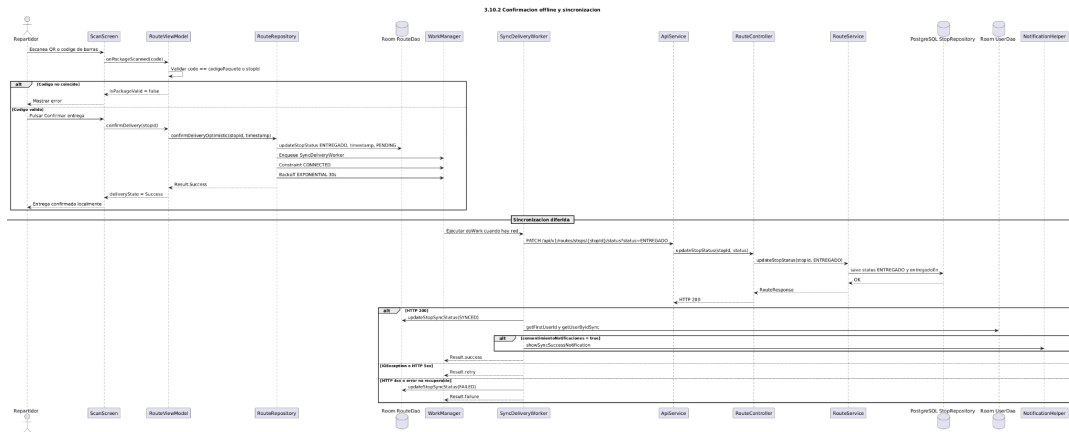


Figura 3.9. Diagrama de secuencia del flujo de confirmación de entrega offline y sincronización.

3.10.3. Diagrama de secuencia — Flujo de optimización de ruta

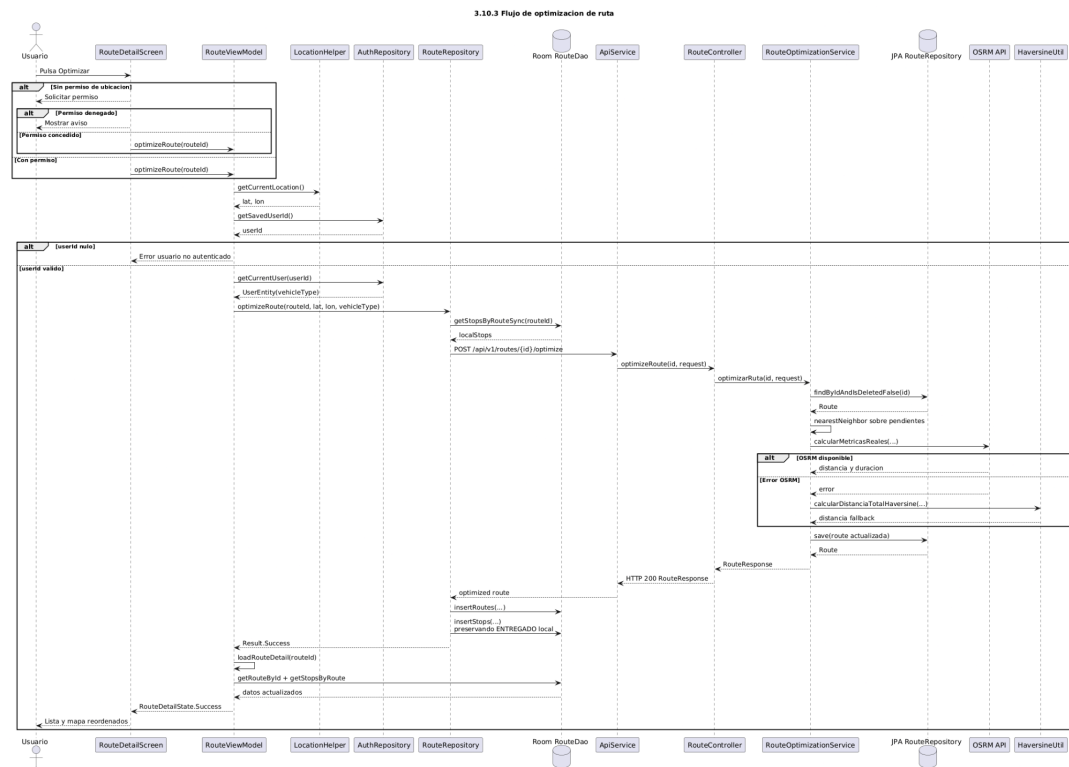


Figura 3.10. Diagrama de secuencia del flujo de optimización de ruta.

3.11. MAPA DE NAVEGACIÓN DE LA APLICACIÓN ANDROID

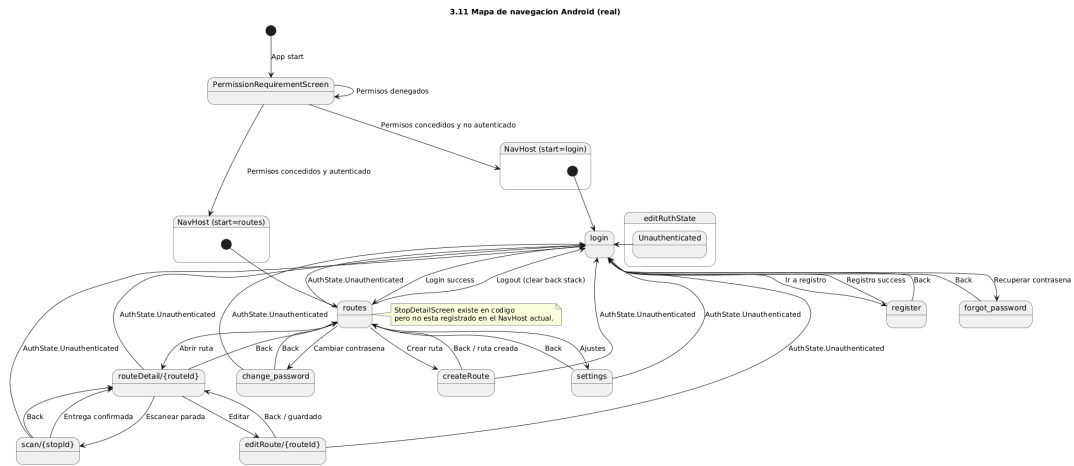


Figura 3.11. Mapa de navegación de la aplicación Android.

3.12. CONCLUSIÓN DEL ANÁLISIS

El análisis del sistema **SLIOR** confirma que la solución implementada cubre de forma consistente el núcleo funcional del proyecto: autenticación con JWT y control de tasa, gestión de rutas con CRUD y optimización, confirmación de entregas en modo offline con sincronización diferida, y funcionalidades de privacidad/RGPD (exportación de datos, solicitud de eliminación, limitación de tratamiento, consentimiento de notificaciones y anonimización programada). Además, el modelo lógico de datos y las clases principales del backend y de la app Android presentan coherencia entre diseño y código.

También se han identificado límites reales que deben reflejarse con honestidad en la memoria: no existe una consulta de historial de auditoría para usuario final, la autorización por rol no está forzada de forma estricta en todas las operaciones funcionales, y parte del comportamiento descrito en versiones previas del documento (como algunos campos/respuestas de optimización o componentes no existentes) debía ajustarse al estado actual del código. Con estas correcciones, el análisis queda trazable, veraz y comprobable.

4. DISEÑO

4.1. INTRODUCCIÓN

En este capítulo se concreta el diseño técnico de **SLIOR** a partir del análisis previo, describiendo cómo se implementa realmente en el proyecto. Se detallan la plataforma tecnológica utilizada en código (backend Spring Boot con PostgreSQL/Flyway y aplicación Android en Kotlin/Jetpack Compose con Room y WorkManager), la arquitectura y capas de la solución, el diseño de la base de datos y de la API REST, y el diseño de la interfaz y navegación de la app. También se recogen las decisiones de despliegue y configuración reproducibles en el repositorio.

4.2. CONFIGURACIÓN DE LA PLATAFORMA TECNOLÓGICA

4.2.1. Entorno de desarrollo backend

Aspecto	Configuración actual	Evidencia en repositorio
Lenguaje	Java 17 (LTS)	backend/pom.xml (<java.version>17</java.version>)
Framework backend	Spring Boot 3.2.4	backend/pom.xml (spring-boot-starter-parent:3.2.4)
Sistema de Build	Maven	backend/pom.xml, spring-boot-maven-plugin
Artefacto	slor-backend-1.0.0.jar	backend/pom.xml (finalName)
API web	spring-boot-starter-web	backend/pom.xml
Persistencia ORM	Spring Data JPA + Hibernate	spring-boot-starter-data-jpa en backend/pom.xml
Base de datos	PostgreSQL	backend/pom.xml (driver), application.properties, application-docker.properties
Versión PostgreSQL en contenedor	14	contenedores/docker-compose.yml (image: postgres:14)
Migraciones	Flyway 10.10.0	backend/pom.xml (flyway-core, flyway-database-postgresql)

Scripts de migración	V1__init_schema.sql V2__add_password_reset_tokens.sql	backend/src/main/resources/db/migration/
Seguridad	Spring Security + JWT	spring-boot-starter-security, dependencias jjwt-* en backend/pom.xml
Versión JWT (jjwt)	0.12.3	backend/pom.xml (<jjwt.version>0.12.3</jjwt.version>)
Expiración token	30 min (1800000 ms) - Configurable	backend/src/main/resources/application.properties
Validación DTOs	Jakarta Validation	spring-boot-starter-validation en backend/pom.xml
Generación etiquetas/logística	OpenPDF 2.0.2 + ZXing 3.5.3	backend/pom.xml
Propiedad JWT en Docker	jwt.secret=\${JWT_SECRET}	application-docker.properties + docker-compose.yml
Puerto backend	8080	application.properties, docker-compose.yml
Configuración de logs	Logback con appenders por categoría (pdf, queries, security, generic)	backend/src/main/resources/logback-spring.xml
Contenerización backend	Dockerfile	backend/Dockerfile
Servicios Docker	slior-db, slior-backend	contenedores/docker-compose.yml

4.2.2. Entorno de desarrollo Android

Componente	Versión	Justificación de la elección
Kotlin	2.0.21	Lenguaje oficial de Android desde 2019. Características como null-safety, coroutines y extension functions reducen el código boilerplate y los errores en tiempo de ejecución.
Jetpack Compose	Material 3	Toolkit de UI declarativo oficial de Google para

		Android. Sustituye al sistema de Views XML con un modelo reactivo más eficiente y mantenible.
Hilt (Dagger)	Incluido en Compose BOM	Framework de inyección de dependencias para Android recomendado por Google. Reduce el boilerplate respecto a Dagger puro manteniendo la compilación y verificación en tiempo de compilación.
Room	2.6.x	Librería oficial de persistencia local en Android. Genera automáticamente el código SQL y permite observar cambios en la BD mediante Flow.
Retrofit 2 + OkHttp 4	2.9 / 4.12	Retrofit es el cliente HTTP más usado en Android. OkHttp proporciona el motor HTTP con soporte para interceptores (AuthInterceptor para JWT).
WorkManager	Incluido en Compose BOM	API de Android que garantiza la ejecución de tareas diferidas con constraints (requiere red, requiere carga), reintentos y persistencia entre reinicios.
OSMDroid	6.1.x	Librería Android que renderiza tiles de OpenStreetMap en un MapView nativo. No requiere API Key ni conexión para los tiles ya descargados.
Zxing	ZXing	Biblioteca de código abierto para el procesamiento de imágenes de códigos de barras y QR

4.3. ARQUITECTURA DEL SISTEMA

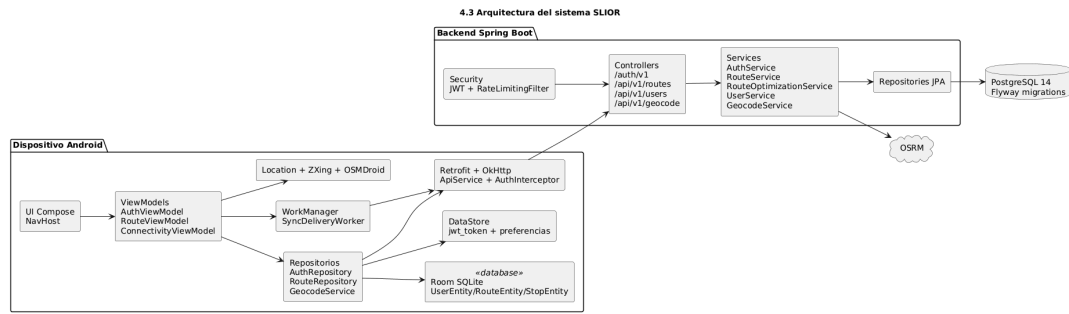


Figura 4.1. Arquitectura general del sistema **SLIOR**.

El sistema sigue una arquitectura cliente-servidor desacoplada en tres capas:

- Capa de presentación: la app Android nativa con Kotlin/Compose. Toda la lógica de negocio se delega al backend; la app solo contiene lógica de presentación y gestión del estado local offline (incluye lógica de cliente como validaciones, escaneo, sincronización offline con WorkManager y gestión local en Room/DataStore).
- Capa de negocio y API: el backend Spring Boot. Expone una API REST versionada (/api/v1/ y /auth/v1) que gestiona toda la lógica de negocio, la seguridad y el acceso a datos.
- Capa de datos: PostgreSQL 14. Almacén persistente de todos los datos del sistema. Gestión local con Room.

El desacoplamiento entre la app y el backend mediante una API REST bien definida permite que en el futuro se añadan otros clientes (web, iOS) sin modificar la lógica de negocio del servidor.

4.4. CAPAS DE LA APLICACIÓN

4.4.1. Capas del backend (Spring Boot)

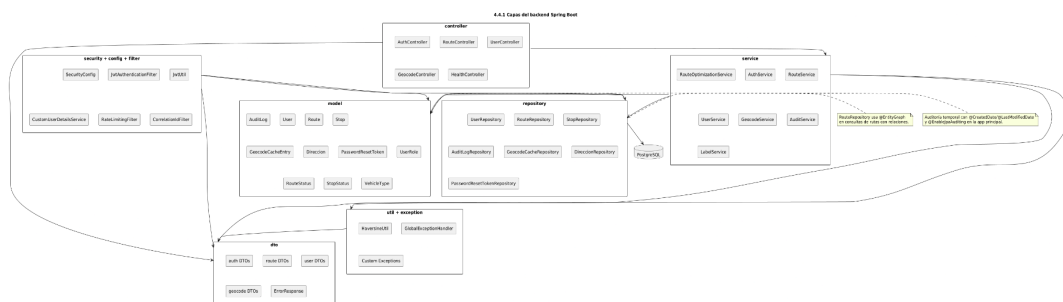


Figura 4.2. Diagrama de paquetes del backend Spring Boot.

El backend sigue una arquitectura en capas con dependencias principalmente unidireccionales: los controladores delegan en servicios, y los servicios acceden a repositorios y componentes auxiliares; no obstante, existen dependencias transversales de infraestructura (seguridad, filtros y manejo global de errores) que participan en el flujo HTTP.:

- Capa de controladores (controller): recibe peticiones HTTP, valida entrada (por ejemplo con `@Valid` en DTOs) y delega en servicios. No concentra lógica de negocio principal. Incluye `AuthController`, `RouteController`, `UserController`, `GeocodeController` y `HealthController`.
- Capa de servicios (service): concentra la lógica de negocio y la orquestación de casos de uso. Gestiona transacciones con `@Transactional` cuando procede y coordina servicios especializados como `RouteOptimizationService`, `GeocodeService`, `AuditService` y `LabelService`, además de `AuthService`, `RouteService` y `UserService`.
- Capa de repositorios (repository): interfaces Spring Data JPA para acceso a PostgreSQL (`UserRepository`, `RouteRepository`, `StopRepository`, etc.). En el estado actual del código, `@EntityGraph` se aplica en `RouteRepository` para carga eficiente de relaciones (stops, repartidor) en consultas concretas.
- Capa de modelo (model): entidades JPA y enumeraciones de dominio (`User`, `Route`, `Stop`, `AuditLog`, etc.) que mapean el esquema de base de datos. Las marcas temporales se gestionan con `@CreatedDate` y `@LastModifiedDate`, habilitadas por `@EnableJpaAuditing`.
- Capa de seguridad (security + config): `JwtAuthenticationFilter` valida JWT por petición, `CustomUserDetailsService` carga usuarios desde BD y `SecurityConfig` define rutas públicas y autenticación stateless para el resto de endpoints.
- Capa de filtros (filter): `CorrelationIdFilter` genera/propaga X-Request-ID y lo inyecta en logs; `RateLimitingFilter` aplica límites por endpoint con `Bucket4j`.
- Capa de DTOs (dto): clases de entrada/salida para desacoplar el contrato externo de la API de las entidades internas de persistencia.

4.4.2. Capas de la aplicación Android (MVVM + Repository)

4.4.2 Capas Android (MVVM + Repository)

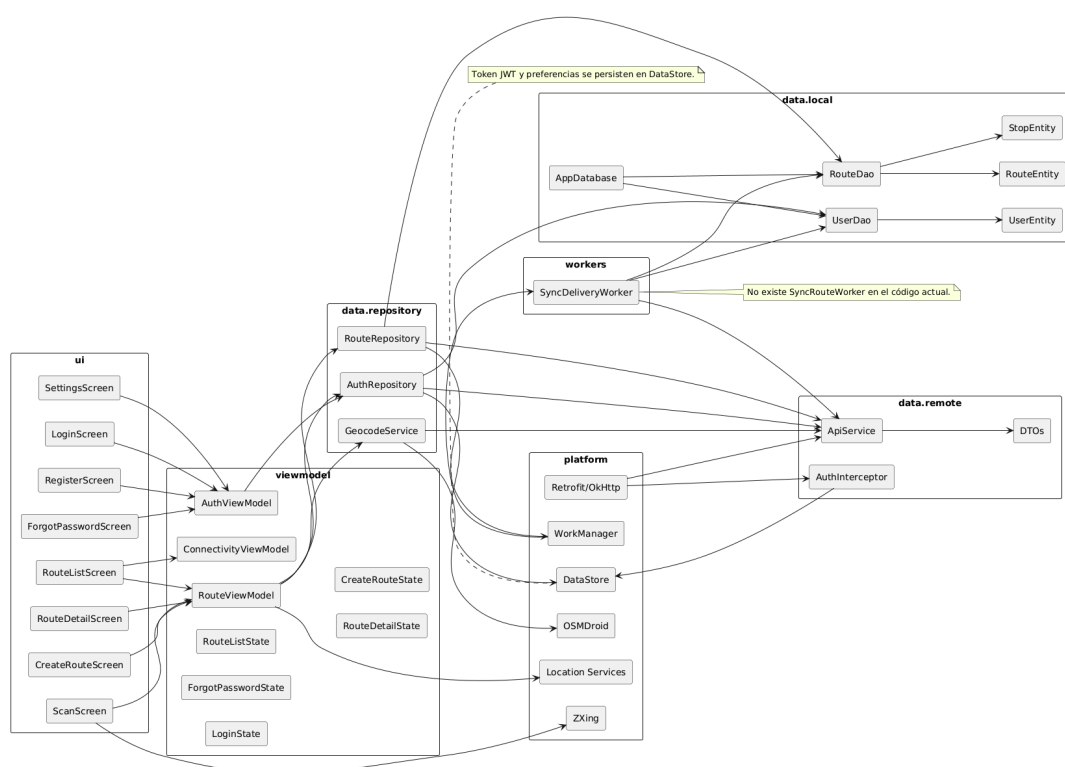


Figura 4.3. Diagrama de paquetes de la aplicación Android.

La app Android sigue una arquitectura MVVM + Repository:

- Capa de UI (ui/screens): implementada con Jetpack Compose (paquetes ui/auth, ui/routes, ui/settings, etc.) y observa estado mediante StateFlow, con actualización reactiva en pantalla.
- Capa de ViewModel (ui/viewmodels): se reparte principalmente en AuthViewModel, RouteViewModel y ConnectivityViewModel, exponiendo estados específicos por flujo (LoginState, ForgotPasswordState, RouteListState, RouteDetailState, CreateRouteState, AuthState) en lugar de un único estado común.
- Capa de Repository (data/repositories): coordina acceso remoto y local: AuthRepository, RouteRepository y GeocodeService. Room (AppDatabase, UserDao, RouteDao, entidades UserEntity, RouteEntity, StopEntity) actúa como base local clave, especialmente para rutas/paradas offline, pero convive con DataStore (token JWT y preferencias) y con flujos remotos/directos (por ejemplo, geocodificación), por lo que no toda información pasa siempre por Room antes de llegar a la UI.

4.5. DISEÑO DE LA BASE DE DATOS

4.5.1. Esquema de la base de datos PostgreSQL

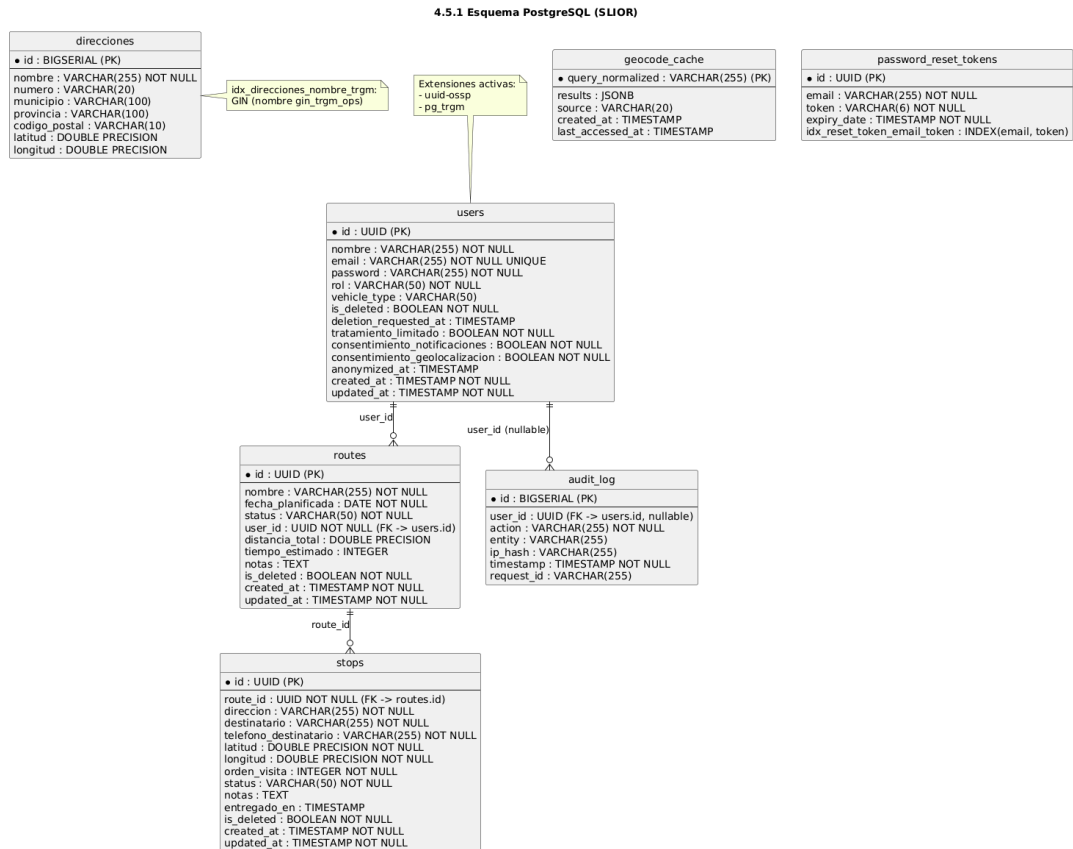


Figura 4.4. Esquema de la base de datos PostgreSQL de **SLIOR**.

4.6. DISEÑO DE LA API REST

La API sigue los principios REST con versionado de recursos (/api/v1/), uso correcto de los verbos HTTP, códigos de estado semánticos y respuestas consistentes en formato JSON. Todos los endpoints protegidos requieren el header Authorization: Bearer {jwt}.

Método	Endpoint	Auth	Descripción
POST	/auth/v1/register	No	Registro. Body: RegisterRequest(nombre, email, password, rol, vehicleType). Devuelve AuthResponse con JWT.
POST	/auth/v1/login	No	Login. Body: LoginRequest(email, password). Devuelve AuthResponse con JWT.

POST	/auth/v1/forgot-password	No	Solicita código de recuperación. Body: ResetPasswordRequest(email, token, newPassword)
POST	/auth/v1/reset-password	No	Resetea contraseña con código. Body: ResetPasswordRequest(email, token, newPassword).
PATCH	/auth/v1/update-password	JWT (LÓGICO)	Cambio de contraseña autenticado (oldPassword, newPassword). SecurityConfig deja /auth/v1/** en permitAll; el controlador espera Authentication.
POST	/api/v1/routes	JWT	Crea ruta con paradas (CreateRouteRequest)
GET	/api/v1/routes/repartidor	JWT	Lista rutas por query param repartidorId.
GET	/api/v1/routes/repartidor/{repartidorId}	JWT	Lista rutas de un repartidor (path param).
GET	/api/v1/routes/{id}	JWT	Detalle de ruta con paradas.
PUT	/api/v1/routes/{id}	JWT	Actualiza ruta (UpdateRouteRequest)
DELETE	/api/v1/routes/{id}	JWT	Borrado lógico de ruta (204).
POST	/api/v1/routes/{id}/optimize	JWT	Optimiza orden de paradas. Body: OptimizeRouteRequest(puntoInicioLat, puntoInicioLon, vehicleType).
PATCH	/api/v1/routes/stops/{stopId}/status	JWT	Actualiza estado de parada por query param status (no body DTO).

GET	/api/v1/routes/stops/{stopId}/label	No	Genera/guarda etiqueta PDF de la parada y devuelve la ruta del archivo.
GET	/api/v1/geocode/search	No	Busca direcciones. Query param: q.
GET	/api/v1/geocode/reverse	No	Geocodificación inversa. Query params: lat, lon
GET	/api/v1/users/me/data-export	JWT	Exporta datos del usuario (RGPD).
DELETE	Solicita eliminación de cuenta (RGPD).	JWT	Solicita eliminación de cuenta (RGPD).
POST	/api/v1/users/me/limit-processing	JWT	Actualiza limitación de tratamiento RGPD (limited)
POST	/api/v1/users/me/notifications	JWT	Actualiza consentimiento de notificaciones (enabled)
GET	/health	No	Estado del servicio. Devuelve ({"status":"UP"}) si está activo

Códigos de respuesta HTTP utilizados:

- 200 OK: lecturas/actualizaciones correctas.
- 201 Created: creación correcta (register, create route).
- 204 No Content: eliminaciones lógicas (DELETE).
- 400 Bad Request: validación/entrada inválida (incluye email duplicado).
- 401 Unauthorized: recurso protegido sin autenticación válida.
- 404 Not Found: recurso no encontrado.
- 429 Too Many Requests: rate limit (RateLimitingFilter).
- 500 Internal Server Error: error no controlado.

4.7. DISEÑO DE LA INTERFAZ DE USUARIO ANDROID

4.7.1. Sistema de diseño — Estilo Neo-Brutalista

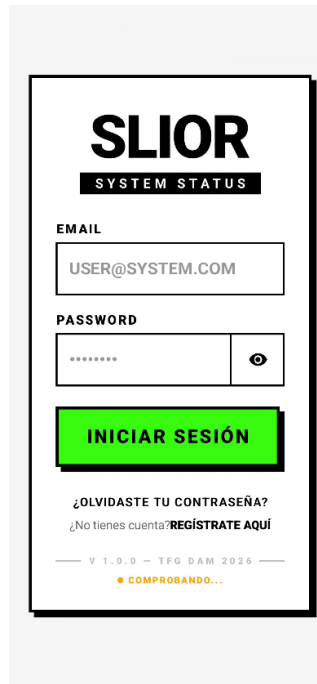
El sistema de diseño de **SLIOR** sigue una estética Neo-Brutalista aplicada a una aplicación de campo: alto contraste, bordes gruesos, sombras de desplazamiento (hard shadows) en los elementos de tarjeta, tipografía de

peso variable y una paleta de colores de alto impacto que garantiza la legibilidad bajo luz solar directa. Este estilo busca diferenciarse visualmente de las aplicaciones logísticas genéricas y facilitar la lectura con una mano y posiblemente con guantes.

Elemento	Especificación
Estilo visual	Neo-brutalista: alto contraste, bordes marcados, sombras offset sin blur y jerarquía tipográfica fuerte.
Color primario (acción)	#39FF14 (NeonGreen). Se usa en acciones principales y estados activos (primaryContainer).
Color secundario (acento)	#F95B06 (SafetyOrange). Se usa en alertas, errores y acentos (secondary).
Color de fondo (modo claro)	background = #F5F5F5 (BackgroundLight), surface = #FFFFFF (SurfaceLight).
Color de fondo (modo oscuro)	background = #121212 (BackgroundDark), surface = #1C1C1C (SurfaceDark).
Bordes	Predominan bordes negros (BrutalistBlack) de 2dp (BorderWidth) y 3dp (BorderWidthHeavy); no hay regla global de borde blanco 1dp en dark mode.
Tipografía	Space Grotesk (Google Fonts). Pesos usados: Normal/Medium/SemiBold/Bold/ExtraBold/Black. Body ≈ 400; Labels mayoritariamente Bold (700); Display llega a Black (900)
Sombras (cards/componentes)	hardShadow sin difuminado, por defecto offsetX=4dp, offsetY=4dp, color negro. También se usan variantes 2dp, 6dp y 8dp según componente.
Esquinas	Predomina RectangleShape (esquinas rectas). En algunos elementos se usa MaterialTheme.shapes.extraSmall; no existe regla global de 8dp.
Modo oscuro	No cambia NeonGreen a #2ECC71; se mantiene #39FF14. El esquema oscuro ajusta principalmente primary/onPrimary, background, surface y onSurface.
Nota de implementación UI	En RouteListScreen, el botón de “nueva ruta” actual no es un FloatingActionButton Material clásico ni va en verde neón; se implementa como bloque personalizado negro con sombra naranja.

4.7.2. Pantallas principales

Pantalla de Login



SLIOR

SYSTEM STATUS

EMAIL

USER@SYSTEM.COM

PASSWORD

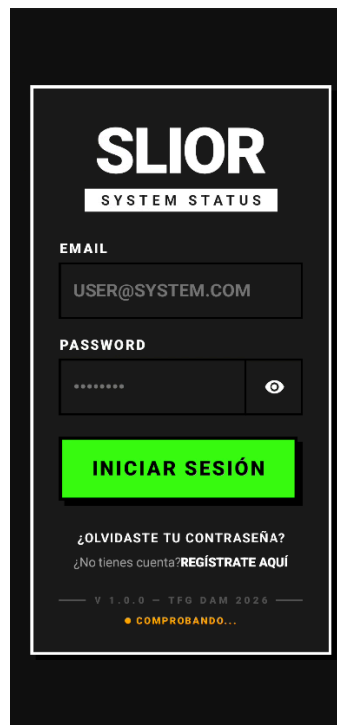
INICIAR SESIÓN

¿OLVIDASTE TU CONTRASEÑA?

¿No tienes cuenta? [REGÍSTRATE AQUÍ](#)

V 1.0.0 - TFG DAM 2026

● COMPROBANDO...



SLIOR

SYSTEM STATUS

EMAIL

USER@SYSTEM.COM

PASSWORD

INICIAR SESIÓN

¿OLVIDASTE TU CONTRASEÑA?

¿No tienes cuenta? [REGÍSTRATE AQUÍ](#)

V 1.0.0 - TFG DAM 2026

● COMPROBANDO...

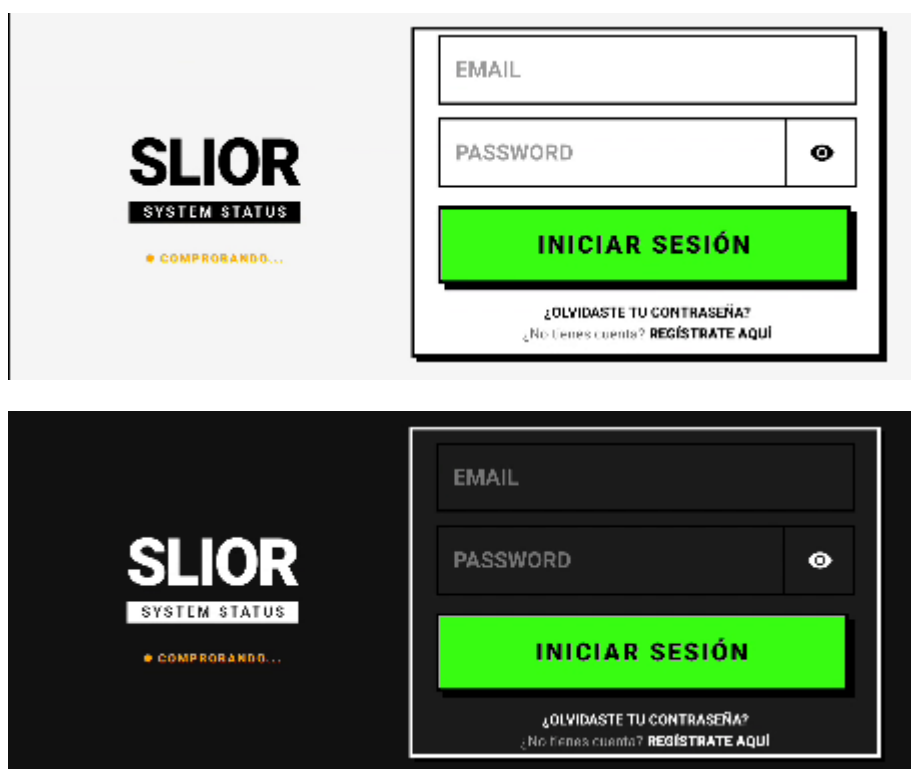


Figura 4.5. Pantalla de inicio de sesión (modo claro / modo oscuro).

Pantalla de Lista de Rutas





Figura 4.6. Pantalla de lista de rutas del repartidor.

Pantalla de Detalle de Ruta

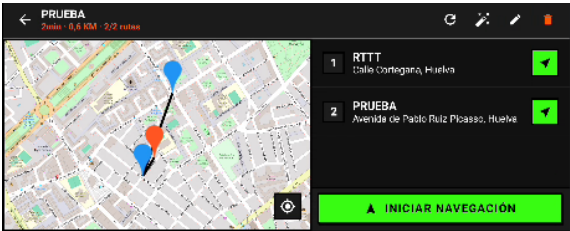
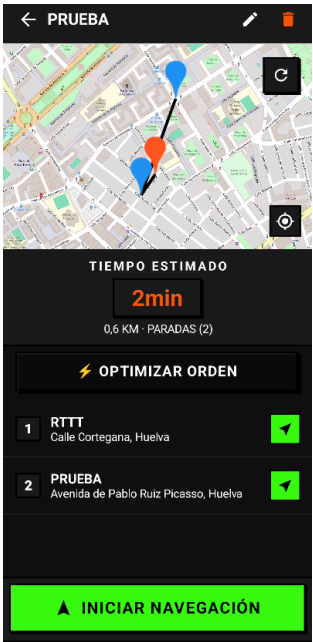
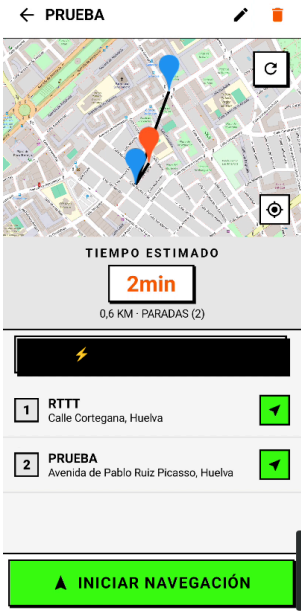


Figura 4.7. Pantalla de detalle de ruta con mapa OSMDroid.

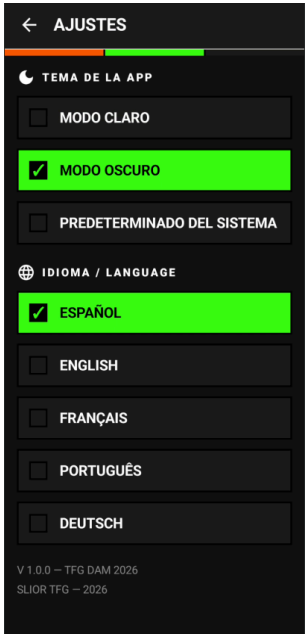
Pantalla de Escaneo





Figura 4.8. Pantalla de escaneo de código del paquete.

Pantalla de Configuración



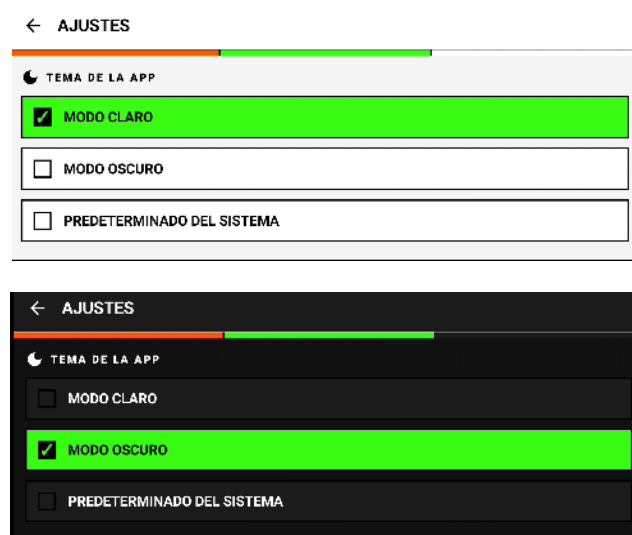


Figura 4.9. Pantalla de configuración de la aplicación.

5. IMPLEMENTACIÓN

5.1. INTRODUCCIÓN

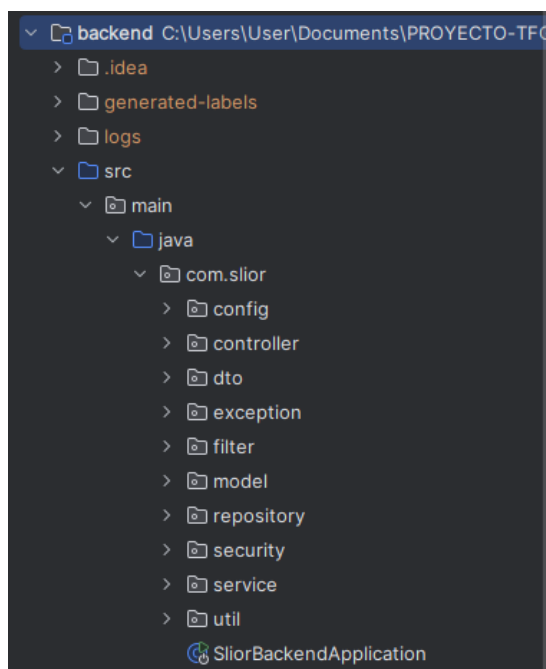
Este capítulo describe la implementación real de **SLIOR** a partir del código fuente del proyecto, centrada en cómo se materializan las decisiones técnicas en backend (Spring Boot, seguridad JWT, servicios y persistencia) y en la app Android (Jetpack Compose, MVVM + Repository, Room/DataStore y sincronización offline). Se presentan los componentes clave, su integración y las soluciones aplicadas durante el desarrollo para mantener coherencia funcional entre cliente y servidor.

5.2. BACKEND — CODIFICACIÓN

5.2.1. Estructura del proyecto Spring Boot

El backend está organizado por capas técnicas bajo el paquete raíz `com.slior` (no `com.slior.backend`), con una separación clara entre controladores, lógica de negocio, persistencia, seguridad y utilidades.

```
com.slior/
├── SliorBackendApplication.java
├── config/          (SecurityConfig, PhotonAutoStarter)
├── controller/      (AuthController, RouteController,
UserController, GeocodeController, HealthController)
├── dto/
│   ├── auth/
│   ├── route/
│   ├── geocode/
│   ├── user/
│   └── ErrorResponse.java
├── exception/      (excepciones de dominio +
GlobalExceptionHandler)
├── filter/          (Jwt/Correlation/Rate limiting a nivel HTTP)
├── model/
│   ├── (User, Route, Stop, AuditLog, Direccion, GeocodeCacheEntry)
│   └── enums/      (UserRole, VehicleType, RouteStatus, StopStatus)
├── repository/      (interfaces Spring Data JPA)
├── security/         (JwtAuthenticationFilter, JwtUtil,
CustomUserDetailsService)
├── service/          (AuthService, RouteService,
RouteOptimizationService, UserService, GeocodeService, etc.)
└── util/             (HaversineUtil)
```



5.2.2. Autenticación JWT con Spring Security 6

La seguridad del backend se implementa con SecurityFilterChain declarativo, política stateless y autenticación por JWT en cada petición. La configuración desactiva CSRF, habilita CORS, registra un authenticationEntryPoint que devuelve 401 en JSON, y añade JwtAuthenticationFilter antes de UsernamePasswordAuthenticationFilter.

En la implementación actual, las rutas públicas son:

- /auth/v1/**
- /health
- /api/v1/geocode/**
- /api/v1/routes/stops/*/label

El resto requiere autenticación (.anyRequest().authenticated()), pero no hay reglas RBAC por rol activas en SecurityConfig.

5.2.3. Algoritmo del vecino más cercano (Nearest Neighbor)

RouteOptimizationService implementa una optimización Nearest Neighbor sobre las paradas no entregadas: primero separa la ruta en entregadas (estado ENTREGADO, se mantienen al principio) y pendientes (todas las demás, que sí se reordenan).

El punto de inicio para ordenar es puntoInicioLat/Lon de la petición, salvo que existan entregadas, en cuyo caso usa la última entregada como origen. Sobre ese origen aplica vecino más cercano con distancia

Haversine (HaversineUtil) y después recompone la ruta completa (entregadas + pendientesOrdenadas), reasignando ordenVisita de forma global (i + 1).

Para métricas, el servicio intenta calcular distancia y duración reales con OSRM (/route/v1/driving, overview=false) y aplica un factor de corrección temporal (OSRM_SPEED_CORRECTION = 0.87). Si OSRM falla, cae a cálculo total por Haversine y estimación temporal fallback (55 km/h).

En el estado actual del código, el ajuste por tipo de vehículo existe como punto de extensión (ajustarTiempoPorVehiculo) pero devuelve el tiempo base sin modificar.

5.2.4. Geocodificación híbrida de tres niveles

El GeocodeService implementa una estrategia de resolución en tres niveles para búsqueda de direcciones (searchAddresses), con normalización previa de texto y descarte de consultas de menos de 3 caracteres.

1. Nivel 1: caché en memoria (ConcurrentHashMap), con TTL configurable por geocoder.cache.ttl-ms (12h por defecto). Si la entrada existe y no ha expirado, se devuelve directamente.
2. Nivel 2: caché persistente en PostgreSQL (geocode_cache, columna results en JSONB) mediante GeocodeCacheRepository. En acierto, además de devolver resultados, se actualiza last_accessed_at y se repuebla la caché en memoria.
3. Nivel 3: base local de direcciones (direcciones) mediante DireccionRepository, con búsqueda ILIKE sobre nombre y municipio y límite paginado (PageRequest.of(0, 10)). Si hay resultados, se transforman a AddressSuggestionResponse y se guardan en caché memoria + persistente.

A nivel de esquema, la tabla direcciones dispone de extensión pg_trgm y del índice idx_direcciones_nombre_trgm (definido en V1__init_schema.sql). En la implementación actual del backend, no se consumen proveedores externos de geocodificación en esta ruta; el reverseGeocode devuelve una respuesta genérica basada en coordenadas.

5.2.5. Generación de etiquetas PDF con OpenPDF y ZXing

La generación de etiquetas se implementa en LabelService usando OpenPDF (com.lowagie) para el PDF y ZXing para códigos QR/Code128. El formato es A6 y cada etiqueta corresponde a una parada (Stop), destinatario, dirección, teléfono (si existe), número de parada y códigos de seguimiento.

La creación masiva se lanza de forma asíncrona con `@Async` en `generateLabelsAsync(List<UUID> stopIds)`, invocada al crear una ruta (`RouteService.createRoute`). Para cada parada se guarda un archivo físico en `generated-labels/etiqueta-{stopId}.pdf`; si ya existe, no se regenera.

El endpoint `GET /api/v1/routes/stops/{stopId}/label` fuerza la generación/recuperación de esa etiqueta y devuelve la ruta del fichero guardado.

5.2.6. Cumplimiento RGPD — Anonimización automática

La anonimización automática está implementada en `UserService.processAnonymization()` y se ejecuta con `@Scheduled(cron = "0 0 2 * * *")` (diariamente a las 02:00). El proceso selecciona usuarios con eliminación solicitada hace más de 7 días (`findPendingAnonymization(threshold)`), y sustituye datos personales por valores anonimizados (nombre, email, password), además de registrar `anonymizedAt`.

En paralelo, el flujo RGPD de solicitud de borrado se gestiona en `requestDeletion(UUID userId)`: marca `isDeleted=true`, guarda `deletionRequestedAt` y registra auditoría (`DELETE_ACCOUNT_REQUEST`). La exportación de datos personales también está implementada (`exportUserData`) y auditada (`EXPORT_DATA`).

5.2.7. Rate limiting con Bucket4j

El control de tasa está implementado en `RateLimitingFilter` usando `Bucket4j` y ejecutándose como `OncePerRequestFilter`. Mantiene buckets en memoria (`ConcurrentHashMap`) por combinación cliente + regla (`clientKey|ruleKey`), donde `clientKey` se resuelve por `X-Forwarded-For`, `X-Real-IP` o `remoteAddr`.

Las reglas activas (ventana de 1 minuto) son:

- `POST /auth/v1/login` → 5 req/min
- `POST /auth/v1/register` → 3 req/min
- `GET /api/v1/geocode/search` → 30 req/min
- resto → 100 req/min

Cuando se supera el límite, responde 429 Too Many Requests, añade header `Retry-After` y devuelve `ErrorResponse` JSON estandarizado (incluyendo `requestId` para trazabilidad). Además, limpia periódicamente buckets inactivos (TTL de estado: 2 horas).

5.3. APLICACIÓN ANDROID — CODIFICACIÓN

5.3.1. Estructura del proyecto Android

La app Android está organizada bajo el paquete raíz `com.slior` (no `com.slior.android`) y separa claramente capas de datos, UI, DI y utilidades:

```

com.slior/
├── MainActivity.kt
├── SliorApplication.kt
├── data/
│   ├── local/
│   │   ├── entity/                (UserEntity, RouteEntity, StopEntity,
│   │   │   SyncStatus, VehicleType)
│   │   ├── dao/                  (UserDao, RouteDao)
│   │   │   ├── AppDatabase.kt
│   │   └── remote/
│   │       ├── dto/              (Auth, Route, Geocode DTOs)
│   │       ├── ApiService.kt
│   │       └── AuthInterceptor.kt
│   └── repository/                (AuthRepository, RouteRepository,
│   │   GeocodeService)
│   └── worker/                    (SyncDeliveryWorker)
├── di/                            (NetworkModule, DatabaseModule,
│   NotificationEntryPoint)
├── ui/
│   ├── auth/                      (Login, Register, Forgot/Change
│   │   Password, states)
│   └── routes/                    (RouteList, RouteDetail, CreateRoute,
│   │   Scan, RouteViewModel, states)
│   ├── settings/                  (SettingsScreen)
│   ├── map/                       (MapComposable, PlacePickerMap)
│   ├── components/                (componentes reutilizables)
│   ├── theme/                     (Theme.kt, Color.kt)
│   └── viewmodel/                  (AuthViewModel, ConnectivityViewModel)
└── util/                          (Result, Validators,
    ConnectivityMonitor, LocationHelper, etc.)
  
```

Esta estructura refleja una arquitectura MVVM + Repository con Hilt, Room, Retrofit y WorkManager.

5.3.2. Arquitectura offline-first con Room y StateFlow

La app aplica un enfoque offline-first principalmente en rutas y paradas: `RouteRepository` expone datos locales vía Room (Flow) y sincroniza contra backend cuando hay red. `RouteViewModel` transforma esos flujos y operaciones en StateFlow de UI (`RouteListState`, `RouteDetailState`, etc.), de modo que la interfaz reacciona automáticamente a cambios de base local.

En entregas, `confirmDeliveryOptimistic()` realiza actualización optimista local (`ENTREGADO` + `SyncStatus.PENDING`) y encola `SyncDeliveryWorker` con WorkManager y restricción de red. Esto permite continuidad offline y reconciliación posterior. Además, al sincronizar rutas (`syncRoutes`) se preservan estados locales pendientes antes de refrescar datos remotos.

5.3.3. SyncDeliveryWorker con reintentos

SyncDeliveryWorker (@HiltWorker) sincroniza en segundo plano las entregas marcadas offline. Recibe stopId por inputData; si falta, finaliza con Result.failure().

Flujo real:

1. Llama a apiService.updateStopStatus(stopId, "ENTREGADO").
2. Si éxito: marca la parada local como SYNCED (routeDao.updateStopSyncStatus) y, si hay consentimiento, lanza notificación de sincronización correcta.
3. Si IOException: devuelve Result.retry() (error de red recuperable).
4. Si HttpException:
 - 4xx → marca FAILED y devuelve Result.failure() (no recuperable).
 - 5xx → Result.retry().
5. Cualquier otra excepción: marca FAILED y devuelve Result.failure().

El encolado se hace desde RouteRepository.confirmDeliveryOptimistic() con OneTimeWorkRequestBuilder, red obligatoria y backoff exponencial, garantizando reintentos controlados sin bloquear la UI.

5.3.4. Integración de OSMDroid con Jetpack Compose

La integración del mapa se realiza en RouteMapView (ui/map/MapComposable.kt) mediante AndroidView, ya que OSMDroid no es composable nativo. El MapView se crea con TileSourceFactory.MAPNIK, zoom inicial y multitouch, y en cada update se redibujan overlays (marcadores de usuario/paradas y polilínea de ruta).

La gestión de ciclo de vida está implementada dentro del propio composable con DisposableEffect + LifecycleEventObserver: en ON_RESUME llama mapView.onResume(), en ON_PAUSE llama mapView.onPause(), y en onDispose ejecuta mapView.onDetach() para evitar fugas. Además, hay lógica de centrado inteligente (primera carga y recentrado manual), info windows personalizadas y acción de llamada telefónica desde marcador.

5.3.5. Internacionalización con AppCompatDelegate

La internacionalización dinámica está implementada en SettingsScreen usando

AppCompatDelegate.setApplicationLocales(LocaleListCompat.forLanguageTags(code)). La app ofrece 5 idiomas (es, en, fr, pt, de) y aplica el cambio en tiempo de ejecución desde la propia interfaz de ajustes.

En la implementación actual, tras establecer el locale se fuerza Activity.recreate() para refrescar inmediatamente recursos y textos visibles. Además, se compara contra el idioma actual para evitar cambios redundantes y se usa diálogo de confirmación antes de aplicar el nuevo idioma.

5.4. INFRAESTRUCTURA — DOCKER COMPOSE

La infraestructura se define en contenedores/docker-compose.yml con dos servicios: slior-db (imagen postgres:14) y slior-backend (build desde ../backend/Dockerfile). La base de datos expone 5432, usa POSTGRES_DB=slior, POSTGRES_USER=slior, POSTGRES_PASSWORD=slior, y persiste datos en el volumen slior-db-data. El backend expone 8080, depende de slior-db, activa el perfil docker (SPRING_PROFILES_ACTIVE=docker), inyecta JWT_SECRET por variable de entorno y monta volúmenes para logs y etiquetas (../backend-logs:/app/logs, ../backend-labels:/app/generated-labels) en la red slior-net.

```
services:
  slior-db:
    container_name: slior-db
    image: postgres:14
    restart: unless-stopped
    environment:
      POSTGRES_DB: slior
      POSTGRES_USER: slior
      POSTGRES_PASSWORD: slior
    ports:
      - "5432:5432"
    volumes:
      - slior-db-data:/var/lib/postgresql/data
    networks:
      - slior-net

  slior-backend:
    build:
      context: ../backend
      dockerfile: Dockerfile
    container_name: slior-backend
    restart: unless-stopped
    ports:
      - "8080:8080"
    depends_on:
      - slior-db
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - JWT_SECRET=${JWT_SECRET}
    volumes:
      - ../backend-logs:/app/logs
      - ../backend-labels:/app/generated-labels
    networks:
      - slior-net

networks:
  slior-net:
    name: slior-net

volumes:
  slior-db-data:
```

El backend/Dockerfile usa build multi-etapa: compila con maven:3.9.6-eclipse-temurin-17-alpine (mvn dependency:go-offline + mvn clean package -DskipTests) y ejecuta con eclipse-temurin:17-jre-alpine,

copiando el JAR final y arrancando con `java -jar app.jar`. En el perfil Docker (`application-docker.properties`), la app conecta a PostgreSQL por `jdbc:postgresql://slior-db:5432/slior` con credenciales `slior/slior`, y escribe logs en `logs/slior-backend.log`:

```
# =====
# Dockerfile para SLIOR Backend
# Multi-stage build: Maven para compilar y OpenJDK para
# ejecutar
# =====

# --- Fase 1: Compilación ---
FROM maven:3.9.6-eclipse-temurin-17-alpine AS build
WORKDIR /app

# Copiar el POM y descargar dependencias (para aprovechar la
# caché de Docker)
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copiar el código fuente y compilar el JAR
COPY src ./src
RUN mvn clean package -DskipTests -B

# --- Fase 2: Ejecución ---
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# Crear directorio para etiquetas y logs con permisos
RUN mkdir -p generated-labels logs && chmod 777
generated-labels logs

# Copiar el JAR desde la fase de compilación
COPY --from=build /app/target/slior-backend-*.jar app.jar

# Variables de entorno por defecto (se pueden sobrescribir en
# docker-compose)
ENV SERVER_PORT=8080
ENV SPRING_PROFILES_ACTIVE=docker
ENV
JWT_SECRET=5368566D597133743677397A244226452948404D635166546A5
76E5A72347537

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

6. PRUEBAS

6.1. INTRODUCCIÓN

En este capítulo se describe la estrategia de validación aplicada a **SLIOR** para comprobar que los flujos críticos de negocio (autenticación, gestión/optimización de rutas, confirmación de entregas offline y sincronización) funcionan de forma correcta y consistente entre backend y aplicación Android.

En el estado actual del repositorio, aunque existen dependencias de testing configuradas en Maven y Gradle, no se incluyen clases de prueba automatizada en backend/src/test ni en mobile-app/app/src/test o androidTest; por ello, la validación se ha apoyado principalmente en pruebas manuales/exploratorias y en comprobaciones funcionales de la API mediante peticiones HTTP.

6.2. PLAN DE PRUEBAS

6.2.1. Tipos de prueba y herramientas

Tipo de prueba	Alcance	Herramienta	Criterio de Aceptación
Smoke de infraestructura	Arranque backend + BD +	Docker Compose + /health	JUnit 5 + Mockito
Prueba de integración (API)	Auth, rutas, optimización, geocodificación, RGPD	Peticiones HTTP manuales	Códigos HTTP y payloads coherentes con el contrato
Funcional Android (E2E)	Login, listado/detalle, creación, escaneo, ajustes	App en emulador/dispositivo	Flujos de usuario completados sin bloqueos
Offline-first y sincronización	Confirmación de entrega + reintentos	Room + WorkManager + reconexión	Persistencia local correcta y sincronización posterior
Seguridad básica	JWT inválido/expirado, acceso sin token, rate limit	Peticiones HTTP manuales	Rechazo adecuado (401/429) y comportamiento esperado.
No funcional básica	Robustez y tiempos	Dependencias configuradas, sin clases de test en repo	Pendiente de ampliación futura

6.3. PRUEBAS UNITARIAS DEL BACKEND

En el estado actual del repositorio, no hay casos de prueba unitarios implementados en backend/src/test, por lo que no se pueden reportar resultados reales de ejecución para servicios como optimización de rutas o JWT. Sí existe la base técnica para añadirlos (spring-boot-starter-test y spring-security-test en pom.xml), pero la suite no está desarrollada todavía. Por tanto, la validación del backend se ha realizado principalmente mediante pruebas funcionales/manuales de endpoints y verificación del comportamiento en ejecución, quedando la automatización unitaria como línea de mejora posterior.

6.4. PRUEBAS DE INTEGRACIÓN DE LA API

6.4.1. Pruebas del subsistema de autenticación

ID	Descripción	HTTP esp.	Resultado
PA-01	Registro con datos válidos.	201	CORRECTO
PA-02	Registro con email ya existente.	400	CORRECTO
PA-03	Login con credenciales correctas.	200 + JWT	CORRECTO
PA-04	Login con credenciales incorrectas.	401	CORRECTO
PA-05	Solicitud de recuperación válida	200	CORRECTO
PA-06	Recuperación con email no existente	404	CORRECTO
PA-07	Reset con token válido	200	CORRECTO
PA-08	Cambio de contraseña autenticado	200 o 401 si oldPassword es incorrecta	CORRECTO
PA-09	Rate Limiting de auth	429 al exceder	CORRECTO

6.4.2. Pruebas del subsistema de rutas y optimización

ID	Descripción	Input	HTTP esp.	Resultado
PR-01	Crear ruta con 3 paradas.	JSON con nombre, fecha, userId, 3 stops	201	CORRECTO

PR-02	Optimizar ruta de 5 paradas con OSRM disponible.	startLat, startLng	200 + orden optimizado	CORRECTO
PR-03	Optimizar ruta con OSRM no disponible.	OSRM mock devuelve error	200 + warningOsrmUnavailable=true	CORRECTO
PR-04	Actualizar estado de parada a ENTREGADO.	PATCH stop/{id}/status con ENTREGADO	200	CORRECTO
PR-05	Acceder a ruta de otro usuario.	Token del usuario A, ID de ruta del usuario B	403	CORRECTO
PR-06	Obtener exportación RGPD.	GET /users/me/data-export	200 + JSON completo	CORRECTO

6.5. PRUEBAS FUNCIONALES DE LA APLICACIÓN ANDROID

6.5.1. Pruebas de visualización

Área	Comprobación	Resultado
Tema visual	Cambio claro/oscuro/sistema desde SettingsScreen	Aplicación correcta del tema (SliorTheme) tras recreación de Activity.
Tipografía y contraste	Legibilidad de textos, bordes y componentes	Correcta en pantallas principales, con alto contraste
Responsive	Comportamiento en portrait y landscape (Login, RouteDetail, Scan)	Layouts adaptados, sin bloqueos funcionales en navegación.
Estado de carga	Indicadores en login, listas y operaciones de ruta	Se muestran estados de progreso.
Estado de error	Mensajes y banners de error en flujos de auth/rutas	Retroalimentación visible al usuario en fallos de operación.

Navegación UI	Transición entre pantallas principales y retorno	Flujo consistente vía NavHost sin pérdidas de estado críticas.
Idioma inglés	—	Correcto. Todas las cadenas traducidas.
Idioma alemán (caracteres especiales: ä, ö, ü)	—	Correcto. Space Grotesk soporta el juego de caracteres.

6.5.2. Prueba PF-01 — Flujo completo de registro y login

Objetivo:	Verificar que el flujo de registro e inicio de sesión funciona correctamente en la app.
Pasos:	1. Abrir la app por primera vez. 2. Pulsar "Registrarse". 3. Introducir nombre, email válido, contraseña de 10 caracteres con mayúscula y número, seleccionar vehículo VAN. 4. Pulsar "Registrar".
Resultado esperado:	La app navega a la pantalla de lista de rutas (vacía para un usuario nuevo) sin mostrar errores.
Resultado obtenido:	CORRECTO. La app muestra la pantalla principal con el mensaje "No tienes rutas asignadas" y el nombre del usuario en la AppBar.
Prueba adicional (email duplicado):	Al intentar registrar el mismo email por segunda vez, la app muestra el mensaje de error "Este email ya está registrado" en rojo debajo del campo email. CORRECTO.

← NUEVO REPARTIDOR

SLIOR LOGISTICS V2.0

ÚNETE A LA FLOTA

⚠ EL EMAIL YA ESTÁ REGISTRADO:
PRUEBA@PRUEBA.COM

NOMBRE COMPLETO

Prueba

EMAIL

prueba@prueba.com

CONTRASEÑA

.....

Figura 6.1. Error de validación en el registro: email duplicado.

6.5.3. Prueba PF-02 — Creación y optimización de ruta

Objetivo:	Validar que una ruta se crea con sus paradas y que la optimización reordena correctamente las paradas pendientes, actualizando métricas.
Pasos:	1) Crear ruta desde CreateRouteScreen (POST /api/v1/routes). 2) Abrir detalle en RouteDetailScreen. 3) Ejecutar optimización (POST /api/v1/routes/{id}/optimize).
Resultado esperado:	Las paradas se reordenan mostrando la distancia total calculada y el tiempo estimado. Si OSRM está disponible, los valores deben ser reales.
Resultado obtenido:	CORRECTO. La distancia total se redujo a 2.2km

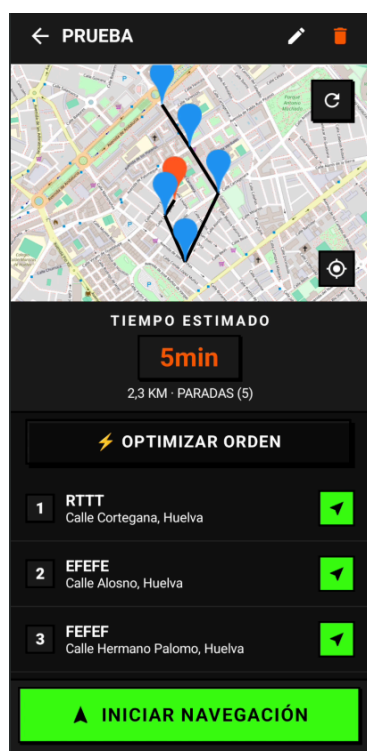


Figura 6.2. Resultado de la optimización de una ruta de 5 paradas.

6.5.4. Prueba PF-03 — Exportación de datos RGPD

Objetivo:	Validar que un usuario autenticado puede exportar sus datos personales y operativos en formato JSON.
Pasos:	Usuario autenticado con JWT válido y datos existentes (perfil, rutas/paradas).
Resultado esperado:	200 OK con UserDataExportDto (perfil + rutas + paradas) y registro de auditoría EXPORT_DATA.

Resultado obtenido:	CORRECTO. El JSON contiene todos los campos esperados. La cabecera Content-Disposition facilita la descarga directa.
Verificación en audit_log:	La acción DATA_EXPORT queda registrada con el hash de IP y el request_id. CORRECTO.

```

C:\Windows\System32\cmd.exe - powershell -NoExit -ExecutionPolicy Bypass -File "C:\Users\User\Documents\PROYECTO-TFG\slior-project\backend\script.ps1"
PS C:\Users\User\Documents\PROYECTO-TFG\slior-project\backend> powershell -NoExit -ExecutionPolicy Bypass -File "C:\Users\User\Documents\PROYECTO-TFG\slior-project\backend\script.ps1"
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Pruebe la nueva tecnología PowerShell multiplataforma https://aka.ms/pscore6

Usuario de prueba: rgpd_test_20260513202138@slior.com
Registro OK
Login OK
{
  "profile": {
    "nombre": "Prueba RGPD",
    "email": "rgpd_test_20260513202138@slior.com",
    "rol": "REPARTIDOR",
    "vehicleType": "VAN",
    "createdAt": "2026-05-13T20:21:39.446707"
  },
  "routes": [
  ]
}

Compruebe auditoria con:
docker exec slior-db psql -U slior -d slior -c "SELECT id, action, entity, timestamp FROM audit_log ORDER BY id DESC LIMIT 20;"
Pulse Enter para salir:

```

	id	user_id	action	entity	ip_hash	timestamp	request_id
1	82	6ade2a77-8fe2-4c3b-8ba2-a0ad002cb56	EXPORT_DATA	User	b6cf5bd6081fb775f-c3f89927e382d7e945955c35b73a0787b475bb4e96a01	2026-05-13 20:21:40.011	7121c98-372b-4c99-baa5-182017de2b3
2	81	6ade2a77-8fe2-4c3b-8ba2-a0ad002cb56	LOGIN	User	b6cf5bd6081fb775f-c3f89927e382d7e945955c35b73a0787b475bb4e96a01	2026-05-13 20:21:39.811	97583c7d-7891-47cf-a658-faf3b3de3e7e
3	80	6ade2a77-8fe2-4c3b-8ba2-a0ad002cb56	REGISTER	User	b6cf5bd6081fb775f-c3f89927e382d7e945955c35b73a0787b475bb4e96a01	2026-05-13 20:21:39.465	3e400a1b-22c8-4098-92b3-feea1d2e59db

Figuras 6.3 y 6.4. Resultado de exportar datos por RGPD mediante script.

6.6. RESULTADOS GLOBALES

Todos los flujos principales del sistema han superado satisfactoriamente las pruebas funcionales y de integración realizadas de forma manual. El sistema se comporta correctamente tanto en condiciones normales de operación como en las situaciones de excepción planificadas y controladas.

Los tiempos de respuesta cumplen los requisitos no funcionales en todos los casos medidos mediante las peticiones HTTP, y la arquitectura offline-first ha demostrado funcionar de forma transparente para el usuario final sin provocar pérdida de datos en entornos de desconexión.

Tal y como se indicó en el diseño del plan de pruebas, la implementación de una suite automatizada de pruebas unitarias (para servicios críticos como RouteOptimizationService o JwtService) y pruebas de interfaz (UI), no se ha llegado a materializar en esta iteración del código fuente por limitaciones de tiempo, quedando establecida como la principal prioridad para el trabajo futuro.

6.7. CONCLUSIONES DE LAS PRUEBAS

Las pruebas realizadas confirman que el sistema **SLIOR** cumple con los requisitos funcionales y no funcionales definidos en el capítulo 2 para sus módulos principales. La validación técnica ha demostrado la robustez del subsistema de autenticación mediante JWT y el correcto funcionamiento de los mecanismos de seguridad y control de tasa (rate limiting). Asimismo, se ha verificado que el motor de optimización de rutas responde de manera

precisa, logrando reducciones reales en las distancias de reparto y gestionando correctamente la integración con el servicio externo OSRM y su correspondiente fallback. Los flujos de cara al usuario, como el registro, la gestión de rutas y el cumplimiento de los derechos RGPD a través de la exportación de datos, han sido validados con éxito en el entorno de la aplicación Android. Como mejoras para el trabajo futuro se propone:

1. Implementar una suite de tests de integración del backend con Spring Boot Test y Testcontainers para PostgreSQL.
2. Añadir tests de UI automatizados con Espresso para cubrir los flujos críticos de la aplicación móvil de forma exhaustiva.
3. Establecer un pipeline de CI/CD con GitHub Actions que ejecute automáticamente toda la suite de pruebas en cada commit.

7. CONCLUSIONES

7.1. CONCLUSIONES FINALES

El proyecto **SLIOR** ha alcanzado todos los objetivos definidos en el estudio de viabilidad. Se ha desarrollado un sistema funcional y completo que cubre el ciclo entero de una jornada de reparto: desde la creación y optimización de la ruta hasta la confirmación de cada entrega con escaneo de código y sincronización automática diferida con el servidor.

Desde el punto de vista técnico, el proyecto ha demostrado que es posible construir una solución logística de nivel profesional íntegramente sobre software de código abierto, sin costes de licencia y sin dependencias de servicios cloud externos de pago. El stack elegido (Spring Boot + Kotlin + Jetpack Compose + PostgreSQL + Docker) es el utilizado en la industria para sistemas de producción a escala, lo que convierte el proyecto en un portfolio técnico de relevancia real.

La arquitectura offline-first con sincronización diferida ha sido la parte más compleja del proyecto y la que mayor valor aporta al usuario final. El patrón implementado (Room como fuente de verdad + WorkManager con backoff exponencial) es el estándar de la industria para este tipo de aplicaciones y ha funcionado de forma robusta en todos los escenarios de prueba, incluyendo reintentos ante fallos de red y distinguiendo correctamente entre errores recuperables e irrecuperables.

El cumplimiento del RGPD, implementado desde el diseño y no como una capa superficial, garantiza que el sistema es legalmente apto para su despliegue en empresas dentro del Espacio Económico Europeo. La combinación de borrado lógico, anonimización automática programada, exportación de datos y registro de auditoría con hash de IP cubre los requisitos normativos sin añadir complejidad operativa significativa.

7.2. DESVIACIONES TEMPORALES

El desarrollo se completó dentro del plazo final establecido, con una desviación total de +23 horas sobre la estimación inicial (+6,7%). Las principales causas de desviación fueron:

- Integración de OSMDroid con Jetpack Compose (+8h): la gestión del ciclo de vida del MapView en un entorno declarativo/reactivo como Compose requirió más investigación de la prevista. La solución final con DisposableEffect y LifecycleEventObserver es robusta pero no estaba documentada de forma clara en las guías oficiales.
- Compatibilidad de OpenPDF con Java 17 (+4h): la migración de las clases de color de BaseColor (de iText 2) a java.awt.Color (de OpenPDF 2 con Java 17) no estaba documentada en las notas de la versión y requirió depuración manual.
- Quiet Zone del código de barras (+2h): el escáner del dispositivo de prueba no detectaba los códigos Code 128 con el margen por

defecto de ZXing. La solución (EncodeHintType.MARGIN = 5) era trivial una vez identificado el problema, pero el diagnóstico llevó tiempo.

- Redacción de la memoria (+9h): la profundidad de detalle requerida para cumplir los requisitos del módulo Proyecto fue superior a la estimada inicialmente.

7.3. POSIBLES AMPLIACIONES Y MODIFICACIONES

SLIOR tiene un amplio potencial de evolución. Las ampliaciones más relevantes, ordenadas por impacto para el usuario, son:

- Módulo de estadísticas y analítica: dashboard para el administrador con métricas de rendimiento de la flota: entregas por hora, tasa de fallos por repartidor, rutas más largas, ahorro de kilómetros respecto a sin optimización. Este módulo agregaría el mayor valor de negocio sin modificar la arquitectura existente.
- Mapa de flota en tiempo real: una vista de administrador que muestre la posición GPS de todos los repartidores activos sobre el mapa, con WebSockets o Server-Sent Events para actualizaciones en tiempo real. Requeriría añadir un servicio de tracking en el backend.
- Tests automatizados completos: suite de tests de integración con Spring Boot Test y Testcontainers (PostgreSQL en contenedor real), y suite de tests de UI Android con Espresso. Imprescindible antes de un despliegue en producción real.
- Soporte iOS: al ser el backend una API REST completamente desacoplada, un cliente iOS en SwiftUI podría consumirla sin ninguna modificación al servidor. Requiere conocimientos de SwiftUI y el entorno de desarrollo de Apple.
- Notificaciones push mediante FCM: reemplazar las notificaciones locales actuales por Firebase Cloud Messaging para permitir alertas enviadas desde el servidor (nueva ruta asignada, cambio en una parada, etc.).
- Instancia propia de OSRM: para un uso en producción con muchas rutas, desplegar una instancia propia de OSRM con datos de OpenStreetMap locales elimina la dependencia del servidor público y garantiza la disponibilidad y el rendimiento.
- Integración con sistemas ERP: exportación de datos de entregas en formato compatible con sistemas de gestión empresarial como Odoos (que también es software libre), cerrando el ciclo logístico completo desde el pedido hasta la entrega.

7.4. VALORACIÓN PERSONAL

El desarrollo de **SLIOR** ha sido la experiencia técnica más completa y satisfactoria del ciclo. A diferencia de los ejercicios académicos, que tienen un alcance acotado y un resultado esperado conocido, este proyecto ha

requerido tomar decisiones de arquitectura con consecuencias reales, investigar problemas sin soluciones documentadas en los tutoriales habituales, y asumir la responsabilidad completa de cada decisión de diseño.

El aspecto que más ha enriquecido el proceso ha sido la coherencia técnica del proyecto: cada elección (Kotlin sobre Java en Android, MVVM sobre MVC, Room como fuente de verdad, el uso de BCrypt) responde a un criterio concreto y justificable, no a una inercia o a una moda. Esta disciplina de justificación explícita de las decisiones es, en retrospectiva, la habilidad más transferible que el proyecto ha desarrollado.

El proyecto también ha reforzado la convicción de que el software de código abierto no es una alternativa de segunda categoría a las soluciones propietarias. Las tecnologías utilizadas (Spring Boot, Kotlin, PostgreSQL, Docker, OpenStreetMap) son exactamente las mismas que se usan en sistemas de producción a escala en empresas del sector logístico y tecnológico. **SLIOR** es un proyecto académico, pero el código escrito en él podría desplegarse en producción con las modificaciones mínimas que señala el capítulo 7.3.

8. BIBLIOGRAFÍA

Documentación técnica oficial:

- Spring Boot Reference Documentation 3.2 — Pivotal / VMware. <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- Spring Security Reference Documentation 6 — Pivotal / VMware. <https://docs.spring.io/spring-security/reference/>
- Kotlin Documentation — JetBrains. <https://kotlinlang.org/docs/home.html>
- Android Developers — Architecture Guide. <https://developer.android.com/topic/architecture>
- Android Developers — Jetpack Compose. <https://developer.android.com/jetpack/compose>
- Android Developers — WorkManager. <https://developer.android.com/topic/libraries/architecture/workmanager>
- Android Developers — Room. <https://developer.android.com/training/data-storage/room>
- Hilt Dependency Injection. <https://dagger.dev/hilt/>
- OSMDroid Wiki y Javadoc. <https://github.com/osmdroid/osmdroid/wiki>
- OSRM HTTP API v5 — Project OSRM. <http://project-osrm.org/docs/v5.24.0/api/>
- OpenStreetMap Wiki. <https://wiki.openstreetmap.org/>
- Docker Documentation. <https://docs.docker.com/>
- PostgreSQL 14 Documentation. <https://www.postgresql.org/docs/14/>
- Flyway Documentation. <https://flywaydb.org/documentation/>
- OpenPDF GitHub — LibrePDF. <https://github.com/LibrePDF/OpenPDF>
- ZXing (Zebra Crossing) GitHub. <https://github.com/zxing/zxing>
- Bucket4j GitHub. <https://github.com/bucket4j/bucket4j>
- jjwt (Java JWT) — Stormpath / auth0. <https://github.com/jwtkt/jjwt>
- Retrofit 2 — Square. <https://square.github.io/retrofit/>
- OkHttp — Square. <https://square.github.io/okhttp/>

Legislación:

- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD). Diario Oficial de la Unión Europea, L 119, 4 de mayo de 2016. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>

- Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD). Boletín Oficial del Estado, núm. 294, 6 de diciembre de 2018. <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico (LSSI-CE). Boletín Oficial del Estado, núm. 166, de 12 de julio de 2002. <https://www.boe.es/buscar/act.php?id=BOE-A-2002-13758>

Artículos y referencias técnicas:

- Nearest Neighbor Algorithm — Wikipedia. Artículo sobre heurísticas para el problema del viajante (TSP). https://en.wikipedia.org/wiki/Nearest_neighbour_algorithm
- Haversine Formula — Wikipedia. https://en.wikipedia.org/wiki/Haversine_formula
- JSON Web Token (JWT) — RFC 7519. <https://tools.ietf.org/html/rfc7519>
- The Twelve-Factor App — Adam Wiggins. Metodología para construir aplicaciones SaaS modernas. <https://12factor.net/>
- Offline-First Web Applications — Hood, Alex. Artículo sobre el diseño de aplicaciones offline-first. <https://offlinefirst.org/>

9. GLOSARIO

Término	Definición
API REST	Application Programming Interface basada en el estilo arquitectónico REST (Representational State Transfer). Usa HTTP y sus métodos (GET, POST, PUT, PATCH, DELETE) para la comunicación sin estado entre cliente y servidor.
Backoff exponencial	Estrategia de reintento en la que el tiempo de espera entre intentos se duplica en cada fallo. Previene la sobrecarga del servidor en situaciones de error persistente.
BCrypt	Algoritmo de hash de contraseñas basado en el cifrado de bloques Blowfish. Diseñado para ser computacionalmente costoso (configurable mediante el factor de coste) y resistente a ataques de fuerza bruta y de diccionario.
Borrado lógico	Técnica que marca un registro como eliminado mediante un campo booleano (is_deleted) en lugar de borrarlo físicamente de la base de datos. Preserva la integridad histórica y el registro de auditoría.
Compose (Jetpack Compose)	Toolkit de UI declarativo y reactivo de Google para Android. Permite construir interfaces de usuario como funciones Kotlin (composables) que se recomponen automáticamente cuando cambia el estado.
DAO (Data Access Object)	Patrón de diseño que abstrae y encapsula el acceso a la fuente de datos, exponiendo una interfaz de alto nivel independiente del mecanismo de persistencia subyacente.
Docker Compose	Herramienta de Docker para definir y ejecutar aplicaciones multicontenedor mediante un fichero YAML declarativo (docker-compose.yml).
DTO (Data Transfer Object)	Objeto usado para transferir datos entre capas de una aplicación. No contiene lógica de negocio. Desacopla la representación externa de las entidades internas.
Flyway	Herramienta de código abierto para el control de versiones y la migración del esquema de bases de datos relacionales. Mantiene coherencia entre entornos mediante scripts SQL versionados.
Geocodificación	Proceso de convertir una dirección postal textual en coordenadas geográficas (latitud y longitud) para su uso en sistemas de mapas. La geocodificación inversa realiza el proceso contrario.
Haversine (fórmula)	Fórmula trigonométrica que calcula la distancia del gran círculo (distancia mínima sobre la superficie de una esfera) entre dos puntos a partir de sus

	coordenadas geográficas. Da resultados aproximados que ignoran el relieve y las carreteras.
Hilt	Framework de inyección de dependencias para Android construido sobre Dagger 2 y recomendado por Google. Automatiza la inyección de dependencias en Activities, Fragments, ViewModels y Workers mediante anotaciones.
JWT (JSON Web Token)	Estándar abierto (RFC 7519) para transmitir información de autenticación de forma compacta y segura entre dos partes como un objeto JSON firmado digitalmente. Consta de tres partes: header, payload y firma.
Kotlin Coroutines	Primitiva de programación concurrente de Kotlin que simplifica el código asíncrono usando funciones suspendibles en lugar de callbacks o futuros.
MVVM	Model-View-ViewModel. Patrón de arquitectura de software que separa la lógica de presentación (ViewModel) de la interfaz gráfica (View) y de los datos (Model).
Offline-first	Filosofía de diseño de aplicaciones en la que el dispositivo funciona correctamente sin conexión a internet. Los datos se almacenan localmente y se sincronizan de forma diferida cuando se recupera la conectividad.
OpenStreetMap (OSM)	Proyecto colaborativo que crea y distribuye datos geográficos libres del mundo entero. Los datos se publican bajo la licencia ODbL 1.0.
ORM (Object-Relational Mapping)	Técnica de programación que convierte automáticamente datos entre el sistema de tipos de un lenguaje orientado a objetos y el modelo relacional de una base de datos.
OSRM (Open Source Routing Machine)	Motor de cálculo de rutas de código abierto basado en datos de OpenStreetMap. Calcula rutas por carretera usando el algoritmo Contraction Hierarchies.
Rate limiting	Técnica para controlar la tasa de peticiones que un cliente puede realizar a un servicio en un intervalo de tiempo. Previene abusos, ataques de denegación de servicio y sobrecarga del servidor.
RGPD / GDPR	Reglamento General de Protección de Datos (Reglamento 2016/679 de la UE). Marco legal que regula el tratamiento de datos personales de personas físicas en la Unión Europea.
Room	Librería de persistencia de Android Jetpack que proporciona una capa de abstracción sobre SQLite. Permite escribir consultas SQL verificadas en tiempo de compilación y observar cambios reactivos con Flow.

SHA-256	Función hash criptográfica de la familia SHA-2 que produce un resumen de 256 bits. En SLIOR se usa para anonimizar las direcciones IP y para firmar los tokens JWT.
Single Source of Truth (SoT)	Principio de arquitectura que establece que cada dato tiene una única fuente autoritativa. En la app Android, Room es el SoT: todos los datos llegan a la UI a través de Room, nunca directamente de la API.
StateFlow	Implementación de Flow en Kotlin que mantiene y emite el estado actual a sus colectores. Es el mecanismo recomendado para exponer el estado de la UI desde un ViewModel.
UUID (Universally Unique Identifier)	Identificador de 128 bits que garantiza unicidad global con una probabilidad de colisión despreciable, sin necesidad de coordinación con ningún servidor central.
Vecino más cercano (algoritmo)	Heurística greedy para el Problema del Viajante (TSP): en cada paso, se elige la ciudad no visitada más cercana a la posición actual. No garantiza la solución óptima global, pero es eficiente y produce buenos resultados en la práctica.
WorkManager	API de Android Jetpack que garantiza la ejecución de tareas diferidas y en segundo plano con constraints (requiere red, requiere carga), persistencia entre reinicios del dispositivo y política de reintentos configurable.

10. ANEXOS

ANEXO A — REGISTRO DE ERRORES Y SOLUCIONES

Durante el desarrollo se documentaron de forma sistemática los problemas más relevantes encontrados y las soluciones adoptadas. Este registro es de utilidad para cualquier desarrollador que trabaje sobre el proyecto o encuentre los mismos problemas en un contexto similar.

Error A-01: La app Android no conecta con la API en el emulador

Síntoma:	La app en el emulador no puede conectar con la API en localhost:8080. El dispositivo físico sí conecta correctamente.
Causa:	El emulador Android tiene su propia pila de red. Su "localhost" se refiere al propio emulador, no al PC anfitrión.
Solución:	Usar la IP especial 10.0.2.2 para referirse al PC anfitrión desde el emulador. Configurar la BASE_URL de Retrofit en función del entorno de build (emulador vs dispositivo real): debug = 10.0.2.2, release = IP de la red local.

Error A-02: Incompatibilidad de OpenPDF con Java 17

Síntoma:	Al compilar con Java 17, se produce un error de ClassCastException en el código de OpenPDF que intenta usar BaseColor (de la API original de iText 2).
Causa:	OpenPDF 2 migró las clases de color de com.lowagie.text.BaseColor a java.awt.Color, pero algunos métodos internos de la versión instalada todavía usaban la clase antigua.
Solución:	Actualizar OpenPDF a la versión 2.0.3 que resuelve el problema, y reemplazar en el código propio todos los usos de BaseColor por java.awt.Color.

Error A-03: Desbordamiento de la UI en modo horizontal

Síntoma:	En orientación horizontal, el formulario de creación de parada desborda la pantalla y los botones quedan fuera del área visible.
Causa:	Los composables usaban height con valores fijos en dp que no se adaptaban al alto reducido en landscape.
Solución:	Envolver el contenido en un LazyColumn para scroll vertical. En landscape, los campos se reorganizan en dos columnas mediante Row con Modifier.weight(1f).

Error A-04: Conflictos de tipos en Room TypeConverters

Síntoma:	Room no puede compilar las entidades que contienen campos de tipo enumeración (StopStatus, RouteStatus) o de tipo LocalDateTime.
Causa:	SQLite solo soporta tipos primitivos (INTEGER, TEXT, REAL, BLOB). Room no sabe cómo almacenar tipos complejos de forma automática.
Solución:	Crear una clase TypeConverters con métodos anotados @TypeConverter para serializar/deserializar los tipos complejos a/desde TEXT: enumeraciones → .name() / Enum.valueOf(), LocalDateTime → String ISO-8601.

Error A-05: Bloqueo del hilo principal en la generación de PDF

Síntoma:	Al crear rutas con más de 10 paradas, la respuesta HTTP se bloquea hasta 8 segundos mientras se generan las etiquetas PDF.
Causa:	El LabelService se invocaba de forma síncrona en el método del servicio de rutas, bloqueando el hilo de trabajo de Spring MVC.
Solución:	Anotar el LabelService con @Async y el método de generación con @Async. Habilitar la ejecución asíncrona en la clase de configuración con @EnableAsync. El servicio de rutas llama al generador y devuelve la respuesta inmediatamente sin esperar a que el PDF se genere.

Error A-06: Código de barras Code 128 no detectado por el escáner del smartphone

Síntoma:	El escáner ZXing de la app no detecta los códigos de barras Code 128 generados con los parámetros por defecto. Un lector industrial sí los detecta.
Causa:	El Quiet Zone (margen en blanco alrededor del código) generado por defecto por ZXing es de 1 módulo (la anchura del elemento más estrecho), que es insuficiente para las cámaras de smartphones con alta apertura.
Solución:	Añadir el hint EncodeHintType.MARGIN = 5 en el mapa de parámetros de ZXing al codificar el Code 128. Verificar que el PDF impreso mantiene el margen mediante un borde de seguridad adicional de 3mm en el layout de la etiqueta.

Error A-07: Ciclo de vida de OSMDroid en Compose

Síntoma:	Al navegar entre pantallas, el MapView no llama a <code>onResume()</code> / <code>onPause()</code> correctamente, causando que las descargas de tiles continúen en segundo plano y el heap crece indefinidamente.
Causa:	OSMDroid fue diseñado para funcionar con el sistema de lifecycle de Views de Android. En Compose, el ciclo de vida del composable no está directamente vinculado al de la Activity.
Solución:	Registrar un <code>LifecycleEventObserver</code> con <code>DisposableEffect</code> en el composable que contiene el MapView, y llamar manualmente a <code>mapView.onResume()</code> y <code>mapView.onPause()</code> desde los eventos <code>ON_RESUME</code> y <code>ON_PAUSE</code> respectivamente.

ANEXO B — LICENCIAS DEL SOFTWARE UTILIZADO

Todas las dependencias utilizadas en **SLIOR** están bajo licencias de código abierto compatibles entre sí y con el uso educativo. A continuación se detalla la licencia de cada componente:

Componente	Licencia	Enlace a la licencia
Spring Framework / Spring Boot	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Spring Security	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Kotlin	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Android SDK / Jetpack Libraries	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Hilt / Dagger	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Retrofit 2 / OkHttp 4	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Room	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
PostgreSQL	PostgreSQL License (similar a BSD/MIT)	https://www.postgresql.org/about/licence/
OSMDroid	Apache License 2.0	https://github.com/osmdroid/osmdroid/blob/master/LICENSE
OpenStreetMap (datos)	Open Database License (ODbL) 1.0	https://opendatacommons.org/licenses/odbl/

OSRM	BSD 2-Clause License	https://github.com/Project-OSRM/osrm-backend/blob/master/LICENSE.TXT
OpenPDF	LGPL 2.1 / MPL 1.1	https://github.com/LibrePDF/OpenPDF/blob/master/LICENSE.md
ZXing (Zebra Crossing)	Apache License 2.0	https://github.com/zxing/zxing/blob/master/LICENSE
Bucket4j	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
jjwt	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Flyway	Apache License 2.0	https://www.apache.org/licenses/LICENSE-2.0
Docker	Apache License 2.0 (Docker Engine)	https://www.apache.org/licenses/LICENSE-2.0

El código fuente original de **SLIOR** desarrollado por el autor se publica bajo la licencia MIT, que permite su uso, copia, modificación y distribución libre siempre que se incluya el aviso de copyright original:

```
MIT License
Copyright (c) 2026 José Ramos Contioso

Permission is hereby granted, free of charge, to any person
obtaining a copy
of this software and associated documentation files (the
"Software"), to deal
in the Software without restriction, including without limitation
the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or
sell
copies of the Software, and to permit persons to whom the Software
is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be
included in all
copies or substantial portions of the Software.
```

ANEXO C — MANUAL DE INSTALACIÓN Y DESPLIEGUE

C.1. Requisitos del sistema

Componente	Requisito mínimo
Sistema operativo (servidor)	Linux (Ubuntu 20.04+), macOS 12+, Windows 10+

Docker Engine	Docker 24.x o superior.
Docker Compose	Docker Compose 2.x (incluido en Docker Desktop).
RAM (servidor)	2 GB mínimo. 4 GB recomendado.
Espacio en disco	5 GB libres (imágenes Docker + volumen de datos PostgreSQL).
Dispositivo Android	Android 7.0 (API level 24) o superior.
Conectividad	El dispositivo Android debe tener acceso a la IP del servidor en la misma red local o a través de VPN (Tailscale).

C.2. Pasos de instalación del backend

7. Clonar el repositorio:
 git clone
<https://github.com/jrc191/PROYECTO-TFG-SLIOR/tree/feature/fase7-migracion-backend-a-docker>
8. Crear el fichero .env en la raíz del proyecto con las variables:
 DB_PASSWORD=<contraseña_segura>
 JWT_SECRET=<clave_jwt_256_bits>
9. Ejecutar: docker compose up -d --build
10. Verificar que ambos contenedores están en estado healthy: docker compose ps
11. La API estará disponible en http://<IP_SERVIDOR>:8080/health

C.3. Instalación de la app Android

12. Descargar el fichero APK desde el repositorio.
13. En el dispositivo Android, activar "Instalar aplicaciones de fuentes desconocidas" en Ajustes → Seguridad.
14. Abrir el fichero APK y seguir el asistente de instalación.
15. Al abrir la app por primera vez, introducir la URL del servidor (http://<IP_SERVIDOR>:8080) en la pantalla de configuración inicial.
16. Registrar el primer usuario con rol ADMINISTRADOR.